Anchored Densest Subgraph Search on Large Bipartite Graphs

Qi Zhang
University of Science and Technology Beijing
Beijing, China
qizhangcs@ustb.edu.cn

Yalong Zhang
Beijing Institute of Technology
Beijing, China
yalong-zhang@qq.com

Rong-Hua Li
Beijing Institute of Technology
Beijing, China
lironghuabit@126.com

Jie He
University of Science and Technology Beijing
Beijing, China
hejie@ustb.edu.cn

Xu-Cheng Yin
University of Science and Technology Beijing
Beijing, China
xuchengyin@ustb.edu.cn

Guoren Wang
Beijing Institute of Technology
Beijing, China
wanggrbit@126.com

ABSTRACT

Bipartite cohesive subgraph mining is a fundamental task for analyzing interactions between two types of entities. Existing bipartite densest subgraph methods optimize global edge concentration under partition-aware normalization, but do not support query-centric exploration. In unipartite graphs, the Anchored Densest Subgraph (ADS) paradigm enables such exploration by requiring the solution to contain a given anchor set while maximizing a reference-aware density objective. However, directly extending this formulation to bipartite graphs with additive normalization does not explicitly capture cross-partition interactions and may therefore yield solutions misaligned with the underlying bipartite structure. In this paper, we formulate the bipartite ADS search problem through a new reference-aware density objective that preserves penalty-based locality and adopts geometric normalization tailored to bipartite graphs. The resulting formulation is a non-linear fractional optimization problem that cannot be handled by standard parametric reductions. To solve it, we develop an aspect-ratio-based parametric framework and establish an equivalence between density feasibility and minimum s-t cut. We further improve scalability through localized feasibility verification around the reference set and convexity-based pruning of the parameter space. Experiments on 16 real-world bipartite graphs demonstrate that our algorithms are efficient and scalable. Case studies further show that the returned subgraphs are structurally coherent and query-relevant.

PVLDB Reference Format:
Qi Zhang, Yalong Zhang, Rong-Hua Li, Jie He, Xu-Cheng Yin, and Guoren Wang. Anchored Densest Subgraph Search on Large Bipartite Graphs. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/qizhang1996/ADSBipartite.git.

1 INTRODUCTION

Bipartite graphs are a fundamental model for representing interactions between two distinct types of entities, such as customer–product engagements in e-commerce [23, 30, 50], author–paper relationships in academia [39, 46, 54], and actor–movie collaborations in the entertainment industry [19, 62]. Mining cohesive subgraphs in bipartite graphs is an important analytical task, as dense interaction patterns often reveal tightly connected groups across the two partitions and support downstream applications such as recommendation [2, 7, 18], research trend analysis [10, 29], and the discovery of tightly collaborating production teams [10, 59]. Consequently, cohesive subgraph discovery in bipartite graphs has been extensively studied in the database and data mining literature.

Existing approaches to bipartite cohesive subgraph mining can be broadly divided into two paradigms. Constraint-based methods characterize cohesiveness through explicit structural requirements, including maximal bicliques [1, 9], k-biplexes [11, 52, 53], k-bitrusses [43, 48, 49, 63], and degree-threshold models such as the (α, β)-core [13, 31, 32]. In contrast, density-maximization methods directly optimize edge concentration under partition-aware normalization. The classical bipartite densest subgraph problem admits polynomial-time exact solutions, but optimizes a purely global objective and therefore returns a single maximizer that may reflect globally prominent structures rather than dense regions centered around specific query vertices. Recent bipartite density decomposition techniques [56] further reveal multi-scale dense structures, yet they remain query-agnostic and do not explicitly enforce locality with respect to a given vertex set. As a result, existing methods still lack a principled density-based formulation for query-centric cohesive subgraph mining in bipartite graphs.

A natural starting point for incorporating locality into density optimization is the Anchored Densest Subgraph (ADS) problem, recently proposed for unipartite graphs [12]. ADS is built on the R-subgraph density, a reference-aware variant of the classical density $2|E(S)|/|S|$, in which each vertex in $S \setminus R$ incurs a penalty equal to its global degree. This design discourages the inclusion of vertices selected merely due to their overall popularity and instead favors subgraphs concentrated around a reference set R. Given an anchor set A and a reference set R with $A \subseteq R$, an ADS is any subgraph that contains A and attains the maximum R-subgraph density. However, directly extending this formulation to bipartite graphs is nontrivial. Retaining the additive denominator $|S|$ normalizes only the total selected size and does not explicitly capture cross-partition interaction, potentially yielding solutions that are less faithful to the underlying bipartite structure. This is undesirable because bipartite cohesiveness fundamentally relies on joint participation from both partitions. These observations motivate the use of geometric normalization $\sqrt{|S^U||S^V|}$, where $|S^U|$ and $|S^V|$ denote the sizes of the selected vertex subsets in the two partitions. Such a normalization

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

is more consistent with the bipartite structure and better captures cross-partition coupling.

Motivated by the above, we formulate the bipartite anchored densest subgraph search problem. Given a bipartite graph $G = (U, V, E)$ and a reference set $R \subseteq U$, we define a new reference-aware density objective that combines geometric normalization with penalties on non-reference vertices. We focus on the practically important setting where the reference set lies on one side of the bipartition, reflecting common query-centric scenarios in which the search starts from a designated entity type and seeks a dense interaction pattern on the other side. By penalizing vertices selected mainly due to global popularity rather than strong association with R, the formulation encourages solutions that are both cross-partition dense and localized around the reference set. At the same time, combining geometric normalization with reference-aware penalties requires a unified treatment of the distinct challenges that arise separately in classical bipartite densest subgraph and unipartite ADS.

Specifically, let $\rho(S) = |E(S)|/\sqrt{|S^U||S^V|}$ denote the classical bipartite density of a bipartite subgraph S. Classical bipartite densest subgraph maximizes $\rho(S)$ and is typically handled through an aspect-ratio-based parametric framework. Our formulation also uses geometric normalization and thus requires explicit reasoning over partition ratios. However, unlike the classical setting, the numerator is further modified by penalties on non-reference vertices, which changes the structure of the fixed-γ feasibility problem. Consequently, the feasibility reduction and associated flow-network construction for classical bipartite densest subgraph do not apply directly. To address this challenge, we introduce an aspect-ratio parameter γ and reformulate the problem into a family of γ-parameterized subproblems. For each fixed γ, we derive a polynomial-time feasibility test by reducing R-subgraph density feasibility to a minimum s-t cut problem in a suitably constructed flow network, which yields a baseline global algorithm.

While this framework establishes tractability, it also exposes two scalability bottlenecks. First, each feasibility test requires constructing and solving a max-flow instance over the entire graph. Second, the aspect-ratio parameter γ may induce a large number of subproblem evaluations. To address the first bottleneck, we exploit a structural confinement property of the flow construction and develop a localized verification procedure that incrementally expands the flow network from the reference set while preserving correctness. To address the second bottleneck, we analyze the scaling behavior of the γ-parameterized objective and derive a safe pruning strategy that substantially reduces the number of γ evaluations without sacrificing optimality.

We summarize our main contributions as follows:

Problem formulation. We formalize the bipartite anchored densest subgraph search problem by combining geometric normalization with a reference-aware density objective under anchor inclusion constraints. This formulation provides a principled way to unify cross-partition balance and query-centric locality.

Novel global algorithm. We develop a global algorithm for bipartite ADS through an aspect-ratio-based parametric reformulation. For each fixed γ, we reduce R-subgraph density feasibility to a minimum s-t cut problem, which yields a baseline algorithm, denoted as GC-ADS. This algorithm also reveals two key scalability bottlenecks: each feasibility test requires solving a max-flow instance over the full graph, and the aspect-ratio search may induce a large number of subproblem evaluations.

Localized verification. We develop LocalVerify, a localized verification procedure that exploits a structural confinement property of the flow construction to restrict computation to the vicinity of the reference set. Replacing global verification with LocalVerify yields a localized algorithm, denoted as LC-ADS, which substantially reduces computational and memory overhead while preserving correctness.

Convexity-based parameter pruning. We establish a convexity property of the R-subgraph density scaling function associated with the γ-parameterized objective. Based on this property, we design a divide-and-conquer pruning strategy that eliminates suboptimal regions of the aspect-ratio search space while preserving optimality. Applied to the global and localized algorithms, this strategy yields two pruned variants, denoted as PGC-ADS and PLC-ADS, and substantially reduces the number of max-flow evaluations.

Extensive experimental evaluation. We conduct experiments on 16 real-world bipartite graphs from diverse domains to evaluate the proposed algorithms. The results show that: (i) PLC-ADS achieves orders-of-magnitude speedups over GC-ADS while consistently returning the same optimal solutions; (ii) localized verification significantly reduces memory consumption compared with global flow construction; (iii) convexity-based pruning dramatically shrinks the effective aspect-ratio search space; and (iv) PLC-ADS scales robustly with respect to graph density and the reference set size $|R|$. In addition, we conduct case studies on dblp-DBDM, imdb-YEAR, and amazon-BOOK, showing that bipartite ADS identifies structurally coherent and query-relevant communities that are not captured by global density objectives. Our source code is publicly available at https://github.com/qizhang1996/ADSBipartite.git.

2 PRELIMINARIES

Consider a bipartite graph $G = (U, V, E)$, where U and V are disjoint vertex partitions and $E \subseteq U \times V$ is the edge set. Let $|U|$, $|V|$, and $|E|$ denote the cardinalities of U, V, and E, respectively. An edge between vertices $u \in U$ and $v \in V$ is denoted by (u, v) or equivalently (v, u). For any vertex $x \in U \cup V$, let $N_G(x) = \{y \in U \cup V \mid (x, y) \in E\}$ denote its neighbor set, and let $d_G(x) = |N_G(x)|$ denote its degree. For a vertex subset $S \subseteq U \cup V$, let $S^U = S \cap U$ and $S^V = S \cap V$ denote the vertices of S restricted to partitions U and V, respectively, and let $\mathrm{Vol}(S) = \sum_{x \in S} d_G(x)$ denote the total degree of vertices in S. The edge set induced by S is denoted by $E(S) = \{(x, y) \in E \mid x \in S,\ y \in S\}$. We denote by $G(S) = (S^U, S^V, E(S))$ the subgraph induced by S, and also use S to denote $G(S)$ when the meaning is clear from context. For disjoint subsets S and T, let $E(S, T) = \{(x, y) \in E \mid x \in S,\ y \in T\}$ denote the set of cross edges between them. Subscripts G are omitted when the context is unambiguous.

We first define the R-subgraph density and then formulate the anchored densest subgraph problem on bipartite graphs.

Definition 2.1. (R-subgraph Density) Given a bipartite graph $G = (U, V, E)$ and a reference set $R \subseteq U$, the R-subgraph density of a vertex subset $D \subseteq U \cup V$ is defined as

$$\rho_R(D) = \frac{2|E(D)| - \sum_{x \in D \setminus R} d_G(x)}{\sqrt{|D^U||D^V|}}. \tag{1}$$

Intuitively, R-subgraph density extends classical bipartite density with a bias toward the reference set R. The penalty term $\sum_{x \in D \setminus R} d_G(x)$ discourages the inclusion of vertices outside R, especially those with high global degrees. Although $\rho_R(D)$ can be negative for an arbitrary vertex subset D, the optimal R-subgraph density

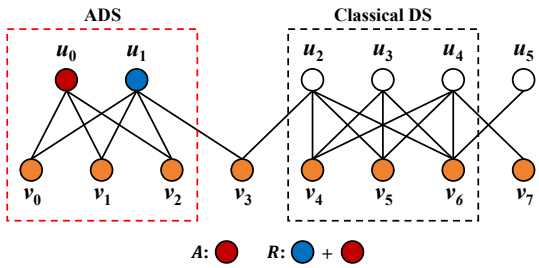


Figure 1: An illustrative example of a bipartite graph G.

is positive whenever $\mathrm{Vol}(R) > 0$, since $\rho_R(G) = \mathrm{Vol}(R)/\sqrt{|U||V|} > 0$ and the optimum is at least $\rho_R(G)$.

Definition 2.2. (Anchored Densest Subgraph, ADS) Given a bipartite graph $G = (U, V, E)$, an anchor set $A \subseteq U$, and a reference set $R \subseteq U$ satisfying $A \subseteq R$, an anchored densest subgraph is a vertex subset $D^* \subseteq U \cup V$ that contains A and maximizes the R-subgraph density, i.e.,

$$D^* = \arg \max_{A \subseteq D,\, D \subseteq U \cup V} \rho_R(D).$$

We use ρ_R^* to denote the optimal R-subgraph density.

Problem Formulation. Given a bipartite graph $G = (U, V, E)$, an anchor set $A \subseteq U$, and a reference set $R \subseteq U$ (where $A \subseteq R$), our goal is to identify the subgraph D^* that maximizes the R-subgraph density and contains the anchor set A. This problem is referred to as the *Anchored Densest Subgraph Search Problem (ADSSP).*

Example 2.3. Consider the bipartite graph G in Figure 1. Let $A = \{u_0\}$ and $R = \{u_0, u_1\}$. The classical densest subgraph is induced by $D = \{u_2, u_3, u_4, v_4, v_5, v_6\}$ (black dotted line), with density $\rho(D) = \frac{9}{\sqrt{3 \cdot 3}} = 3$. In contrast, the anchored densest subgraph (red dotted line) is induced by $D^* = \{u_0, u_1, v_0, v_1, v_2\}$, with R-subgraph density $\rho_R(D^*) = \frac{2 \cdot 6 - 2 \cdot 3}{\sqrt{2 \cdot 3}} = \sqrt{6}$. This example shows that the classical approach does not incorporate bias toward R, and therefore may fail to recover dense subgraphs that are locally relevant to R.

3 A GLOBAL ALGORITHM

In this section, we present GC-ADS, a global algorithm for the ADSSP. Our key idea is to introduce a γ-parameterized variant of the R-subgraph density, referred to as γ-based R-subgraph density, which reformulates the original problem into a family of fixed-γ subproblems. For each such subproblem, we develop a verification procedure based on network flow. By combining this feasibility test with an enumeration over candidate aspect-ratio pairs, we obtain GC-ADS for computing the ADS.

3.1 Parametric Reformulation

The key difficulty in bipartite ADS lies in the geometric normalization term $\sqrt{|D^U||D^V|}$, which couples the two partitions in a non-linear manner. To make the objective more tractable, we adopt an aspect-ratio-based parametric reformulation. Specifically, for a fixed parameter $\gamma > 0$, we define a γ-parameterized variant of the R-subgraph density, which replaces the geometric denominator with a linear surrogate while preserving tightness at the correct aspect ratio.

Definition 3.1. (γ-based R-subgraph Density) Given a bipartite graph $G = (U, V, E)$, a reference set $R \subseteq U$, and a parameter $\gamma > 0$, the γ-based R-subgraph density of a vertex subset $D \subseteq U \cup V$ with $D^U \neq \emptyset$ and $D^V \neq \emptyset$ is defined as

$$\rho_{\gamma,R}(D) = \frac{2\sqrt{\gamma}\sqrt{\gamma'}}{\gamma + \gamma'} \rho_R(D) = \frac{2\sqrt{\gamma}\, w(D)}{\gamma |D^V| + |D^U|}, \tag{2}$$

where $w(D) = 2|E(D)| - \sum_{x \in D \setminus R} d_G(x)$ and $\gamma' = \frac{|D^U|}{|D^V|}$ is the aspect ratio of D.

Intuitively, γ serves as a hypothesized aspect ratio between $|D^U|$ and $|D^V|$. The original R-subgraph density involves a geometric mean in the denominator, whereas $\rho_{\gamma,R}(D)$ replaces it with the weighted arithmetic mean $\gamma|D^V| + |D^U|$, which is linear in the selected vertex sets. As shown below, this surrogate is always a lower bound on $\rho_R(D)$ and becomes tight exactly when $\gamma = |D^U|/|D^V|$.

Definition 3.2. (γ-based Anchored Densest Subgraph) Given a bipartite graph $G = (U, V, E)$, an anchor set $A \subseteq U$, a reference set $R \subseteq U$ satisfying $A \subseteq R$, and a parameter $\gamma > 0$, a γ-based anchored densest subgraph is any vertex subset $D_\gamma^* \subseteq U \cup V$ such that $A \subseteq D_\gamma^*$ and

$$\rho_{\gamma,R}(D_\gamma^*) = \max_{\substack{A \subseteq D,\, D \subseteq U \cup V, \\ D^U \neq \emptyset,\, D^V \neq \emptyset}} \rho_{\gamma,R}(D).$$

We use $\rho_{\gamma,R}^*$ to denote the optimal γ-based R-subgraph density, and refer to the problem of computing D_γ^* as the *γ-based Anchored Densest Subgraph Search Problem* (γ-ADSSP).

Example 3.3. Consider the bipartite graph G in Figure 1. Let $A = \{u_0\}$ and $R = \{u_0, u_1\}$. The ADS is given by $D^* = \{u_0, u_1, v_0, v_1, v_2\}$, with $\rho_R(D^*) = \sqrt{6}$. Since $|D^* \cap U| = 2$ and $|D^* \cap V| = 3$, the actual aspect ratio is $\gamma' = 2/3$. If we set $\gamma = 2$, then the scaling factor $\frac{2\sqrt{2}\sqrt{2/3}}{2+2/3} < 1$, yielding $\rho_{\gamma,R}(D^*) = \frac{3\sqrt{2}}{2} < \rho_R(D^*)$. In contrast, when $\gamma = 2/3$, the scaling factor equals 1, and hence $\rho_{\gamma,R}(D^*) = \rho_R(D^*)$. This example illustrates the lower-bound and tightness properties formalized in the following lemma.

Lemma 3.4. *For any vertex subset $D \subseteq U \cup V$ with $D^U \neq \emptyset$, $D^V \neq \emptyset$, and $\rho_R(D) > 0$, and for any parameter $\gamma > 0$, the following properties hold:*

(1) ***Lower Bound:*** $\rho_{\gamma,R}(D) \leq \rho_R(D)$.
(2) ***Tightness:*** $\rho_{\gamma,R}(D) = \rho_R(D)$ *if and only if* $\gamma = \frac{|D^U|}{|D^V|}$.

Proof. Let $\alpha = \frac{2\sqrt{\gamma}\sqrt{\gamma'}}{\gamma+\gamma'}$. By the AM-GM inequality, $\frac{\gamma+\gamma'}{2} \geq \sqrt{\gamma\gamma'}$, which implies $\alpha \leq 1$, with equality if and only if $\gamma = \gamma'$. Since $\rho_{\gamma,R}(D) = \alpha\, \rho_R(D)$ and $\rho_R(D) > 0$, it follows that $\rho_{\gamma,R}(D) \leq \rho_R(D)$, with equality if and only if $\alpha = 1$, i.e., $\gamma = \gamma'$. □

Lemma 3.4 immediately yields the following theorem, which forms the basis of our global algorithm.

Theorem 3.5. *The maximum R-subgraph density is given by*

$$\rho_R^* = \max_{\gamma > 0} \rho_{\gamma,R}^*.$$

Proof. Let D^* be an ADS, and let $\gamma^* = \frac{|D^{*U}|}{|D^{*V}|}$ denote its aspect ratio. By Lemma 3.4 (Tightness),

$$\rho_R^* = \rho_R(D^*) = \rho_{\gamma^*,R}(D^*).$$

Since $\rho^*_{\gamma^*,R}$ is the maximum γ^*-based R-subgraph density, we have

$$\rho^*_R = \rho_{\gamma^*,R}(D^*) \le \rho^*_{\gamma^*,R} \le \max_{\gamma>0} \rho^*_{\gamma,R}.$$

Conversely, fix any $\gamma > 0$, and let D^*_γ be an optimal solution to the corresponding γ-ADSSP. Since $\rho_{\gamma,R}(G) > 0$ whenever $\mathrm{Vol}(R) > 0$, we have $\rho^*_{\gamma,R} > 0$, and hence $\rho_R(D^*_\gamma) > 0$. Applying Lemma 3.4 (Lower Bound) to D^*_γ gives

$$\rho^*_{\gamma,R} = \rho_{\gamma,R}(D^*_\gamma) \le \rho_R(D^*_\gamma) \le \rho^*_R.$$

Since this holds for every $\gamma > 0$, we obtain

$$\max_{\gamma>0} \rho^*_{\gamma,R} \le \rho^*_R.$$

Combining the two inequalities yields

$$\rho^*_R = \max_{\gamma>0} \rho^*_{\gamma,R}.$$

□

Theorem 3.5 reduces the ADS search problem to solving a family of fixed-γ subproblems. Consequently, our global algorithm consists of two components: enumerating valid aspect ratios γ and, for each fixed γ, computing the optimal value $\rho^*_{\gamma,R}$. Since fixing γ yields a linear denominator in Eq.2, the resulting subproblem becomes amenable to an efficient network-flow-based solution, as detailed in the next subsection.

3.2 Solving γ-ADSSP via Network Flow

In this subsection, we solve the γ-ADSSP via threshold verification. For a threshold g, we ask whether there exists a feasible vertex subset D such that $\rho_{\gamma,R}(D) > g$. We show that this decision problem can be reduced to a minimum s-t cut problem, which in turn enables binary search on the optimal value.

Verification via Min-Cut. We first rewrite the condition $\rho_{\gamma,R}(D) > g$ into a form amenable to minimum-cut computation.

Lemma 3.6. *Let $D \subseteq U \cup V$ be a vertex subset with $D^U \neq \emptyset$ and $D^V \neq \emptyset$, and let $\overline{D} = (U \cup V) \setminus D$. For a threshold $g \ge 0$, the condition $\rho_{\gamma,R}(D) > g$ holds if and only if*

$$\mathrm{Vol}(\overline{D} \cap R) + |E(D,\overline{D})| + |D^U| \cdot \frac{g}{2\sqrt{\gamma}} + |D^V| \cdot \frac{g\sqrt{\gamma}}{2} < \mathrm{Vol}(R). \quad (3)$$

Proof. By Definition 3.1, the condition $\rho_{\gamma,R}(D) > g$ is equivalent to

$$\frac{2\sqrt{\gamma}\, w(D)}{\gamma|D^V| + |D^U|} > g,$$

where $w(D) = 2|E(D)| - \sum_{x\in D\setminus R} d_G(x)$. Rearranging gives

$$2|E(D)| - \sum_{x\in D\setminus R} d_G(x) > |D^U|\frac{g}{2\sqrt{\gamma}} + |D^V|\frac{g\sqrt{\gamma}}{2}.$$

Next, using

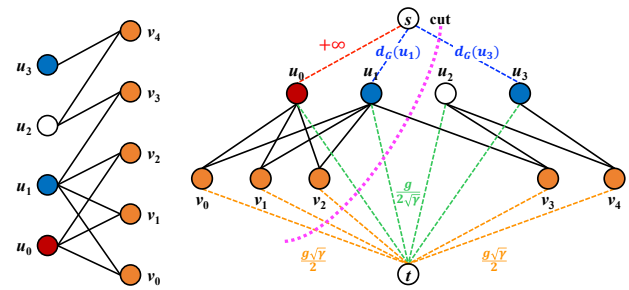
$$\sum_{x\in D} d_G(x) = 2|E(D)| + |E(D,\overline{D})|,$$

we obtain

$$2|E(D)| = \sum_{x\in D} d_G(x) - |E(D,\overline{D})|.$$

Substituting this into the inequality above yields

$$\sum_{x\in D\cap R} d_G(x) - |E(D,\overline{D})| > |D^U|\frac{g}{2\sqrt{\gamma}} + |D^V|\frac{g\sqrt{\gamma}}{2}.$$



(a) The graph G' (b) The flow network $G'_{\gamma,g}$

Figure 2: Construction of the flow network $G'_{\gamma,g}$ from G'.

Since

$$\sum_{x\in D\cap R} d_G(x) = \mathrm{Vol}(R) - \mathrm{Vol}(\overline{D} \cap R),$$

the claim follows immediately. □

Lemma 3.6 implies that, for any vertex subset D, the condition $\rho_{\gamma,R}(D) > g$ holds if and only if Eq.3 is satisfied. Hence, deciding whether there exists a subset D with $\rho_{\gamma,R}(D) > g$ reduces to minimizing the left-hand side of Eq.3 over all admissible D. This minimization can be formulated as an s-t min-cut problem in a suitably constructed flow network. We construct the flow network $G_{\gamma,g} = (V_{\gamma,g}, E_{\gamma,g})$ as follows:

(1) **Nodes:** $V_{\gamma,g} = U \cup V \cup \{s,t\}$.
(2) **Bipartite edges:** For each $(u,v) \in E$, add directed edges $\langle u,v\rangle$ and $\langle v,u\rangle$ with capacity 1.
(3) **Anchor constraints:** For each $u \in A$, add $\langle s,u\rangle$ with capacity $+\infty$ (ensuring $u \in D$ in any finite cut).
(4) **Reference weights:** For each $u \in R \setminus A$, add $\langle s,u\rangle$ with capacity $d_G(u)$.
(5) **Penalty weights:** For each $u \in U$, add $\langle u,t\rangle$ with capacity $\frac{g}{2\sqrt{\gamma}}$; for each $v \in V$, add $\langle v,t\rangle$ with capacity $\frac{g\sqrt{\gamma}}{2}$.

Example 3.7. Consider the bipartite graph G' in Figure 2(a), with anchor set $A = \{u_0\}$ and reference set $R = \{u_0, u_1, u_3\}$. Figure 2(b) shows the corresponding flow network $G'_{\gamma,g}$. For visual clarity, explicit edge directions are omitted: each line between U and V represents a pair of opposing unit-capacity arcs, while all edges incident to s or t are directed as defined above. In particular, s connects only to vertices in R, and the arc to the anchor u_0 has infinite capacity to enforce inclusion.

The following lemma formalizes the connection between minimum s-t cuts in $G_{\gamma,g}$ and the verification condition.

Lemma 3.8. *Let (S,T) be a finite s-t cut in $G_{\gamma,g}$, and let $D = S \setminus \{s\}$ and $\overline{D} = (U \cup V) \setminus D$. Then:*

(1) *The capacity of (S,T) is exactly $\mathrm{Vol}(\overline{D} \cap R) + |E(D,\overline{D})| + |D^U|\frac{g}{2\sqrt{\gamma}} + |D^V|\frac{g\sqrt{\gamma}}{2}$.*
(2) *There exists a vertex subset D such that $\rho_{\gamma,R}(D) > g$ if and only if the minimum s-t cut capacity in $G_{\gamma,g}$ is strictly less than $\mathrm{Vol}(R)$.*

Proof. The cut capacity of a finite s-t cut (S,T) consists of three parts: (1) edges from s to $\overline{D} \cap R$ (capacity $\mathrm{Vol}(\overline{D} \cap R)$); (2) edges

from D to $\overline{D}$ (capacity $|E(D,\overline{D})|$); and (3) edges from D to t (capacity $|D^U|\frac{g}{2\sqrt{\gamma}} + |D^V|\frac{g\sqrt{\gamma}}{2}$). Summing these yields the left-hand side of Eq.3. By Lemma 3.6, a vertex subset D satisfies $\rho_{\gamma,R}(D) > g$ if and only if the corresponding cut capacity is strictly less than $\mathrm{Vol}(R)$. Therefore, such a subset exists if and only if the minimum s-t cut capacity in $G_{\gamma,g}$ is strictly less than $\mathrm{Vol}(R)$. □

Search Space and Termination Condition. To guarantee the exactness of our algorithm in solving the γ-ADSSP, we analyze the search boundaries and derive the termination threshold δ required to distinguish the optimal γ-based R-subgraph density from sub-optimal values.

Lemma 3.9. *(Bounds) The optimal γ-based R-subgraph density satisfies $0 < \rho^*_{\gamma,R} \le \rho^*_R \le \mathrm{Vol}(R)$.*

Proof. First, $\rho^*_R > 0$ holds because $\rho_R(G) = \mathrm{Vol}(R)/\sqrt{|U||V|} > 0$ whenever $\mathrm{Vol}(R) > 0$, and thus the optimum is at least $\rho_R(G)$. For any vertex subset D, we have

$$\rho_R(D) = \frac{2|E(D)| - \sum_{x\in D\setminus R} d_G(x)}{\sqrt{|D^U||D^V|}} \le \frac{\mathrm{Vol}(D\cap R)}{\sqrt{|D^U||D^V|}} \le \mathrm{Vol}(R),$$

which implies $\rho^*_R \le \mathrm{Vol}(R)$.

Let D^*_γ be an optimal solution to the γ-ADSSP. Since

$$\rho^*_{\gamma,R} \ge \rho_{\gamma,R}(G) > 0,$$

we have $\rho_{\gamma,R}(D^*_\gamma) > 0$, and hence $\rho_R(D^*_\gamma) > 0$. Applying Lemma 3.4 to D^*_γ, we obtain

$$\rho^*_{\gamma,R} = \rho_{\gamma,R}(D^*_\gamma) \le \rho_R(D^*_\gamma) \le \rho^*_R.$$

Therefore,

$$0 < \rho^*_{\gamma,R} \le \rho^*_R \le \mathrm{Vol}(R).$$

□

With the search boundaries established, we now determine the termination condition. Crucially, since the input graph is discrete, the possible values of $\rho_{\gamma,R}(D)$ are discrete. The next lemma guarantees that by refining the binary search interval to a sufficient precision, we can exactly recover the γ-based ADS.

Lemma 3.10. *(Gap Lemma) Let $\gamma = \frac{p}{q}$ with $p, q \in \mathbb{Z}_{>0}$. For any two vertex subsets $D_1, D_2 \subseteq U \cup V$ with distinct γ-based R-subgraph density values, the difference satisfies:*

$$\left|\rho_{\gamma,R}(D_1) - \rho_{\gamma,R}(D_2)\right| \ge \frac{2\sqrt{\gamma}}{(p|V| + q|U|)^2}.$$

Proof. Recalling Eq.2, the γ-based R-subgraph density is given by $\rho_{\gamma,R}(D) = \frac{2\sqrt{\gamma}\cdot w(D)}{\gamma|D^V|+|D^U|}$. Substituting $\gamma = p/q$, the denominator becomes $\frac{p|D^V|+q|D^U|}{q}$. Thus, we can rewrite the objective function as:

$$\rho_{\gamma,R}(D) = 2\sqrt{\gamma}\cdot\frac{w(D)\cdot q}{p|D^V| + q|D^U|} = 2\sqrt{\gamma}\cdot\frac{N_D}{Q_D},$$

where $N_D = w(D)\cdot q \in \mathbb{Z}$ and $Q_D = p|D^V| + q|D^U| \in \mathbb{Z}$. The denominator Q_D is bounded by $Q_{\max} = p|V| + q|U|$. Consider two subgraphs with distinct objective values $v_1 = 2\sqrt{\gamma}\frac{N_1}{Q_1}$ and $v_2 = 2\sqrt{\gamma}\frac{N_2}{Q_2}$. The absolute difference is $2\sqrt{\gamma}\left|\frac{N_1Q_2 - N_2Q_1}{Q_1Q_2}\right|$. Since the numerator is a non-zero integer, it is at least 1. Therefore, the difference is at least $\frac{2\sqrt{\gamma}}{Q^2_{\max}}$. □

Algorithm 1: GC-ADS(G, R, A)

Input: Bipartite graph $G = (U, V, E)$, reference set $R \subseteq U$, anchor set $A \subseteq R$.
Output: The anchored densest subgraph D^*.

1 $\rho^*_R \leftarrow 0$; $D^* \leftarrow \emptyset$;
2 **foreach** (n_1, n_2) *such that* $1 \le n_1 \le |U|$ *and* $1 \le n_2 \le |V|$ **do**
3 $\quad\gamma \leftarrow n_1/n_2$;
4 $\quad lb \leftarrow 0$; $ub \leftarrow \mathrm{Vol}(R)$;
5 $\quad\delta \leftarrow \frac{2\sqrt{\gamma}}{(|V|n_1+|U|n_2)^2}$;
6 $\quad$**while** $ub - lb \ge \delta$ **do**
7 $\quad\quad g \leftarrow (lb + ub)/2$;
8 $\quad\quad(V_{\gamma,g}, E_{\gamma,g}) \leftarrow$ BuildFlowNetwork(G, R, A, γ, g);
9 $\quad\quad(S, T) \leftarrow$ MinCut$(V_{\gamma,g}, E_{\gamma,g})$ where $s \in S, t \in T$;
10 $\quad\quad D \leftarrow S \setminus \{s\}$;
11 $\quad\quad$**if** $capacity(S, T) < \mathrm{Vol}(R)$ **then**
12 $\quad\quad\quad lb \leftarrow g$;
13 $\quad\quad\quad$**If** $g > \rho^*_R$ **then** $\{\rho^*_R \leftarrow g; D^* \leftarrow D;\}$
14 $\quad\quad$**else** $ub \leftarrow g$;
15 **return** D^*;
16 **Procedure** BuildFlowNetwork(G, R, A, γ, g)
17 Initialize $V_{\gamma,g} \leftarrow U \cup V \cup \{s, t\}$, $E_{\gamma,g} \leftarrow \emptyset$;
18 Add arcs $\langle s, u\rangle$ with cap $+\infty$ for each $u \in A$;
19 Add arcs $\langle s, u\rangle$ with cap $d_G(u)$ for each $u \in R \setminus A$;
20 Add arcs $\langle u, v\rangle$ and $\langle v, u\rangle$ with cap 1 for each $(u, v) \in E$;
21 Add arcs $\langle u, t\rangle$ with cap $\frac{g}{2\sqrt{\gamma}}$ for each $u \in U$;
22 Add arcs $\langle v, t\rangle$ with cap $\frac{g\sqrt{\gamma}}{2}$ for each $v \in V$;
23 **return** $(V_{\gamma,g}, E_{\gamma,g})$;

Based on Lemma 3.10, setting the binary search threshold $\delta < \frac{2\sqrt{\gamma}}{(p|V|+q|U|)^2}$ guarantees that the final search interval contains at most one valid value of $\rho_{\gamma,R}(D)$. This allows us to unambiguously recover the exact optimal subgraph D^*_γ upon termination.

3.3 The Exact Algorithm

We now present the complete algorithm, GC-ADS, for solving the ADSSP. At a high level, GC-ADS enumerates candidate aspect-ratio pairs and performs binary search for each induced value of γ.

The execution flow of GC-ADS is detailed in Algorithm 1. The algorithm begins by initializing the best feasible lower bound ρ^*_R and the corresponding candidate solution D^* (Line 1). It then enumerates all candidate aspect-ratio pairs (n_1, n_2) and sets $\gamma = n_1/n_2$ for each pair (Lines 2–3). For a fixed γ, the algorithm initializes the binary search interval $[lb, ub]$ and computes the precision threshold δ according to the Gap Lemma (Lines 4–5). Within the binary search loop (Lines 6–7), it tests a threshold value g by constructing the flow network $G_{\gamma,g}$, computing the minimum s-t cut (S, T), and extracting the corresponding vertex subset D (Lines 8–10). If the cut capacity is strictly less than $\mathrm{Vol}(R)$, then g is a feasible lower bound on $\rho^*_{\gamma,R}$; in this case, the algorithm updates lb to g. If g also improves the best feasible lower bound found over all previously examined parameter pairs, it updates the global best value ρ^*_R and stores the corresponding solution D (Lines 11–13). Otherwise, it decreases the upper bound to g (Line 14). After all candidate aspect-ratio pairs have been processed, the algorithm returns the recorded solution D^* (Line 15).

We now formally establish the correctness and computational complexity of GC-ADS.

Theorem 3.11. *Algorithm* GC-ADS *correctly identifies the ADS D^*, runs in $O(n^4\sqrt{m}\log n)$ time, and consumes $O(n+m)$ space, where $n = |U| + |V|$ and $m = |E|$.*

Proof. **Correctness:** The outer loop enumerates all candidate aspect-ratio pairs (n_1, n_2), and therefore covers the optimal aspect ratio $\gamma^* = \frac{|D^{*U}|}{|D^{*V}|}$ of the ADS D^*. For each fixed γ, Lemma 3.8 guarantees the correctness of the feasibility test, and Lemma 3.10 ensures that binary search with threshold δ isolates the unique optimal γ-based R-subgraph density. Hence, GC-ADS correctly returns the ADS D^*.

Time Complexity: The outer loop enumerates $O(|U||V|) = O(n^2)$ candidate aspect-ratio pairs (n_1, n_2). For each pair, binary search performs $O(\log(\frac{\mathrm{Vol}(R)}{\delta})) = O(\log n)$ iterations by Lemma 3.10. Each iteration computes a minimum s-t cut on a flow network with $O(n)$ vertices and $O(m+n)$ edges, which takes $O(n^2\sqrt{m})$ time using Push–Relabel. Therefore, the total running time is $O(n^2 \cdot \log n \cdot n^2\sqrt{m}) = O(n^4\sqrt{m}\log n)$.

Space Complexity: Each flow network has $O(n)$ vertices and $O(m+n)$ edges, and the auxiliary data structures used by Push–Relabel are linear in the network size. Therefore, the total space complexity is $O(n+m)$. □

Bottlenecks and Motivation. Although GC-ADS theoretically guarantees exactness, its time complexity of $O(n^4\sqrt{m}\log n)$ is computationally prohibitive for large-scale graphs. This scalability barrier stems from two fundamental inefficiencies. First, **global verification overhead**: for every instance, the algorithm constructs a flow network and computes the min-cut over the *entire* graph G. This incurs unnecessary overhead, as the anchored densest subgraph typically resides in the local vicinity of the reference set R. Second, **vast parameter space**: the number of candidate aspect ratios γ scales quadratically ($O(n^2)$). Exhaustively enumerating this massive search space—where each step involves an expensive flow computation—is inefficient. Consequently, to achieve scalability, we propose strategies to localize the flow network construction (Section 4) and optimize the parameter search process (Section 5).

4 LOCALIZED VERIFICATION

To reduce the cost of verifying whether $\rho^*_{\gamma,R} > g$, we develop a localized verification algorithm that avoids constructing the full flow network. The key observation is that, in $G_{\gamma,g}$, flow can only be injected from the reference set R, which suggests that the relevant min-cut structure is often confined to a local region around R. Based on this observation, we replace the static global graph with a dynamically expanded local sub-network and show that, under a suitable boundary condition, local verification suffices to certify global correctness.

4.1 Local Flow Network Construction

Our localized construction exploits the structure of the flow network $G_{\gamma,g}$. Since the source s is adjacent only to vertices in the reference set R, the cut relevant to verifying $\rho^*_{\gamma,R} > g$ may often be confined to a local region around R. To capture this structure, we define a local sub-network over an active set $S = R \cup B$, together with its boundary $N(S) = (\bigcup_{u\in S} N_G(u)) \setminus S$, where $B \subseteq (U \cup V) \setminus R$ is an auxiliary set of additional vertices.

Definition 4.1. (Local Flow Network) Given a reference set R and an auxiliary vertex set $B \subseteq (U \cup V) \setminus R$, let $S = R \cup B$. The local flow network $G^B_{\gamma,g} = (V_B, E_B)$ is defined as follows:

(1) **Nodes:** $V_B = \{s, t\} \cup S \cup N(S)$.

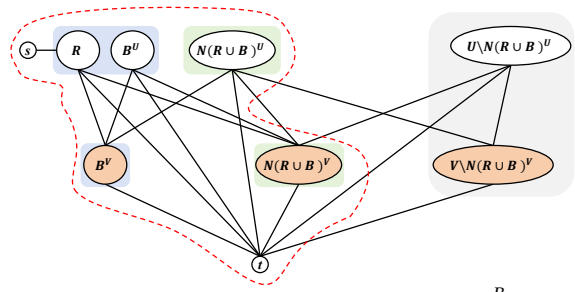


Figure 3: Schematic of the local flow network $G^B_{\gamma,g}$. Vertices are grouped into the active set S, the boundary set $N(S)$, and the outside set, indicated by blue-, green-, and gray-shaded background regions, respectively. The red dashed frame marks the portion of the global network retained in the local construction.

(2) **Edges:** E_B contains exactly the following arcs, with capacities inherited from the global flow network $G_{\gamma,g}$:
- all source arcs $\langle s, u\rangle$ for $u \in R$;
- all arcs between vertices in S;
- all sink arcs $\langle u, t\rangle$ for $u \in S$;
- for each boundary vertex $v \in N(S)$, all arcs from S to v, together with the sink arc $\langle v, t\rangle$.

The local network retains all edges needed to evaluate cuts whose source side is contained in S, while using the boundary $N(S)$ to detect whether the current active region must be further expanded. Figure 3 illustrates this construction. The following lemma shows that, under a suitable boundary condition, the minimum cut found in the local network is already globally optimal.

Lemma 4.2. *Let f be a maximum flow in the local network $G^B_{\gamma,g}$. If none of the sink arcs $\langle v, t\rangle$ for boundary vertices $v \in N(S)$ are saturated by f, then the minimum s-t cut value in $G^B_{\gamma,g}$ is equal to the minimum s-t cut value in the global network $G_{\gamma,g}$. Consequently, the local minimum cut induces a global minimum cut.*

Proof. Let $(S_{\mathrm{loc}}, T_{\mathrm{loc}})$ be a minimum s-t cut in the local network $G^B_{\gamma,g}$. By the max-flow min-cut theorem, every arc crossing from S_{loc} to T_{loc} must be saturated under the maximum flow f.

We first show that no boundary vertex can belong to S_{loc}. Suppose, for contradiction, that there exists a vertex $v \in N(S) \cap S_{\mathrm{loc}}$. By construction, the local network contains the sink arc $\langle v, t\rangle$. Since $t \in T_{\mathrm{loc}}$, this arc crosses the cut and must therefore be saturated, contradicting the assumption. Hence, $N(S) \subseteq T_{\mathrm{loc}}$.

It follows that the source side of the local minimum cut is contained entirely in the active set S. We now extend $(S_{\mathrm{loc}}, T_{\mathrm{loc}})$ to a cut of the global network by assigning every vertex outside V_B to the sink side. Because all boundary vertices already belong to T_{loc}, this extension introduces no additional cut edge. Therefore, the resulting global cut has exactly the same capacity as the local cut, which implies that the global minimum cut value is at most the local minimum cut value.

Conversely, let $(S_{\mathrm{glob}}, T_{\mathrm{glob}})$ be any s-t cut in the global network, and restrict it to the local vertex set V_B. Since the local network is obtained by omitting edges outside V_B, the capacity of the restricted cut in the local network is at most its capacity in the global network. Therefore, the local minimum cut value is at most the global minimum cut value.

Combining the two inequalities, the minimum cut values in the local and global networks are equal. Consequently, the local minimum cut induces a global minimum cut. □

4.2 The Local Expansion Algorithm

Building on Lemma 4.2, we develop LocalVerify, a localized verification procedure that avoids constructing the full flow network. Starting from the reference set R, it expands the active region only when boundary sink arcs become saturated.

To improve efficiency, LocalVerify uses an incremental computation strategy. As the active region expands, the previously computed flow remains feasible in the enlarged local network, because the enlarged network preserves all previously existing arcs and capacities, while newly introduced arcs can be initialized with zero flow. Therefore, instead of recomputing a maximum flow from scratch after each expansion, each iteration only augments the current flow in the updated residual network.

The LocalVerify procedure (Lines 15–27 of Algorithm 2) follows a saturate-and-expand loop:

(1) **Initialization:** Initialize the expansion set as $B \leftarrow \emptyset$ and the current flow as $f \leftarrow 0$. The initial active set is therefore $S = R$.
(2) **Incremental Augmentation:** In each iteration, construct the local network $G^B_{\gamma,g}$ over the current active set $S = R \cup B$ and its boundary $N(S)$. Then augment the current flow by computing additional flow in the residual network.
(3) **Boundary Saturation Test:** Identify the set of saturated boundary vertices $V_{\text{sat}} = \{ v \in N(S) \mid \langle v, t\rangle \text{ is saturated by } f\}$.
(4) **Expansion or Termination:** If $V_{\text{sat}} \neq \emptyset$, expand the active region by setting $B \leftarrow B \cup V_{\text{sat}}$. Otherwise, by Lemma 4.2, the current local minimum cut is already globally optimal, and the procedure terminates.

This saturation-driven strategy restricts exploration to the region necessary for certifying the cut, so the active set S is typically much smaller than the full graph, i.e., $|S| \ll |U \cup V|$.

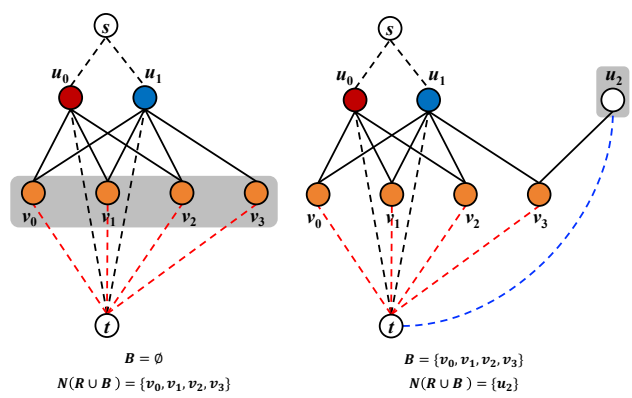
Example 4.3. Figure 4 illustrates the execution of LocalVerify on the bipartite graph shown in Figure 1, with $g = 2$ and $\gamma = 2/3$. Initially, the active set is $S = R = \{u_0, u_1\}$, and its boundary is $N(S) = \{v_0, v_1, v_2, v_3\}$. Here, C_V and C_U denote the capacities of the sink arcs from V-side and U-side vertices to the sink t, respectively; under $\gamma = 2/3$, they evaluate to $C_V \approx 0.82$ and $C_U \approx 1.22$. In the first iteration (Figure 4(a)), the unit flow injected from R saturates all boundary sink arcs, since the inflow of 1.0 exceeds the sink capacity for V-vertices ($C_V \approx 0.82$). These saturated boundary vertices are therefore absorbed into the expansion set, yielding $B \leftarrow \{v_0, v_1, v_2, v_3\}$. In the second iteration (Figure 4(b)), the active region expands accordingly, and the new boundary includes u_2. However, the available flow of 1.0 does not saturate the boundary sink arc $\langle u_2, t\rangle$, because the sink capacity for U-vertices is larger ($C_U \approx 1.22$). Hence $V_{\text{sat}} = \emptyset$, and the procedure terminates with a local minimum cut that is globally optimal by Lemma 4.2.

The LC-ADS **algorithm.** By replacing the global verification step in GC-ADS with LocalVerify, we obtain the complete localized algorithm LC-ADS, shown in Algorithm 2. The algorithm follows the same parametric-search framework for solving the bipartite ADS problem, while using localized verification to test whether $\rho^*_{\gamma,R} > g$ (Line 8). Since LocalVerify returns a globally optimal cut whenever it terminates, LC-ADS preserves the correctness of GC-ADS while substantially reducing unnecessary global exploration.

Algorithm 2: LC-ADS(G, R, A)

Input: Bipartite graph $G = (U, V, E)$, reference set $R \subseteq U$, anchor set $A \subseteq R$.
Output: The anchored densest subgraph D^*.

1 $\rho^*_R \leftarrow 0$; $D^* \leftarrow \emptyset$;
2 **foreach** (n_1, n_2) *such that* $1 \le n_1 \le |U|$ *and* $1 \le n_2 \le |V|$ **do**
3 $\quad \gamma \leftarrow n_1/n_2$;
4 $\quad lb \leftarrow 0$; $ub \leftarrow \text{Vol}(R)$;
5 $\quad \delta \leftarrow \frac{2\sqrt{\gamma}}{(|V|n_1+|U|n_2)^2}$;
6 $\quad$ **while** $ub - lb \ge \delta$ **do**
7 $\quad\quad g \leftarrow (lb + ub)/2$;
8 $\quad\quad (S_{cut}, T_{cut}) \leftarrow \text{LocalVerify}(G, R, A, \gamma, g)$;
9 $\quad\quad D \leftarrow S_{cut} \setminus \{s\}$;
10 $\quad\quad$ **if** $capacity(S_{cut}, T_{cut}) < \text{Vol}(R)$ **then**
11 $\quad\quad\quad lb \leftarrow g$;
12 $\quad\quad\quad$ **if** $\rho_R(D) > \rho^*_R$ **then** $\{\rho^*_R \leftarrow \rho_R(D); D^* \leftarrow D;\}$
13 $\quad\quad$ **else** $ub \leftarrow g$;
14 **return** D^*;
15 **Procedure** LocalVerify(G, R, A, γ, g)
16 $B \leftarrow \emptyset$; $f \leftarrow 0$;
17 **while** *True* **do**
18 $\quad S \leftarrow R \cup B$;
19 $\quad$ Construct the local network $G^B_{\gamma,g}$ on $S \cup N(S)$;
20 $\quad f \leftarrow f + \text{MaxFlow}(\text{Residual}(G^B_{\gamma,g}, f))$;
21 $\quad V_{\text{sat}} \leftarrow \emptyset$;
22 $\quad$ **foreach** $v \in N(S)$ **do**
23 $\quad\quad$ **if** *arc* $\langle v, t\rangle$ *is saturated by* f **then** $V_{\text{sat}} \leftarrow V_{\text{sat}} \cup \{v\}$;
24 $\quad$ **if** $V_{\text{sat}} = \emptyset$ **then break**;
25 $\quad B \leftarrow B \cup V_{\text{sat}}$;
26 $(S_{cut}, T_{cut}) \leftarrow \text{MinCut}(G^B_{\gamma,g})$;
27 **return** (S_{cut}, T_{cut});



(a) Iteration 1 (Expansion) (b) Iteration 2 (Termination)

Figure 4: Illustration of the LocalVerify **procedure on graph** G**. Boundary vertices under inspection are highlighted in gray; red dashed arcs to** t **indicate saturated boundary sink arcs, while blue dashed arcs indicate unsaturated ones.**

4.3 Complexity Analysis

We now establish the time and space complexity of the LocalVerify procedure. Since the algorithm employs an incremental computation strategy, its complexity is determined by the size of the final local flow network $G^{B^*}_{\gamma,g}$ constructed at termination. Therefore, our analysis focuses on bounding the dimensions of this final network and the corresponding max-flow computational cost.

In the following analysis, we use the invariant that once a boundary vertex is absorbed into the expansion set B, the saturation of its sink arc is preserved in all subsequent iterations of LocalVerify. This invariant is naturally maintained by the incremental augmentation strategy, since each expansion only enlarges the local network and the algorithm continues from the current feasible flow in the updated residual network.

We begin with the size of the final expansion set B^*.

Lemma 4.4. $|B^*| \le \frac{2\cdot \mathrm{Vol}^2(R)}{g^2}$.

Proof. Let f^* be the maximum flow in the final local network $G^{B^*}_{\gamma,g}$. For every vertex $u \in B^*$, the sink arc $\langle u, t\rangle$ is saturated at the moment when u is absorbed into B, and remains saturated thereafter by the maintained invariant. Hence, in the final flow f^*, every vertex in B^* contributes its sink capacity.

In addition, every anchor vertex in A must remain on the source side of any finite cut because the arc $\langle s, u\rangle$ has infinite capacity for each $u \in A$. Therefore, the sink arcs of anchor vertices must also be paid in any finite s-t cut, and thus contribute to the max-flow value. We obtain

$$\mathrm{val}(f^*) \ge \sum_{u\in B^*\cup A} \mathrm{cap}(u,t) = (|B^{*U}| + |A|)\frac{g}{2\sqrt{\gamma}} + |B^{*V}|\frac{g\sqrt{\gamma}}{2}. \quad (4)$$

On the other hand, consider the cut $(\{s\}\cup A,\ (U\cup V)\setminus A\cup\{t\})$ in the global flow network. Its capacity is $\mathrm{Vol}(R) + |A|\frac{g}{2\sqrt{\gamma}}$. Since the final local flow value cannot exceed the global minimum cut value, we have

$$\mathrm{val}(f^*) \le \mathrm{Vol}(R) + |A|\frac{g}{2\sqrt{\gamma}}. \quad (5)$$

Combining Eq.4 and Eq.5, and applying the AM-GM inequality, we obtain

$$\mathrm{Vol}(R) \ge 2\sqrt{|B^{*U}|\frac{g}{2\sqrt{\gamma}}\cdot |B^{*V}|\frac{g\sqrt{\gamma}}{2}} = g\sqrt{|B^{*U}||B^{*V}|}.$$

Squaring both sides yields

$$|B^{*U}||B^{*V}| \le \frac{\mathrm{Vol}^2(R)}{g^2}.$$

Assuming the non-trivial case where $|B^{*U}| \ge 1$ and $|B^{*V}| \ge 1$, we have $|B^{*U}| \le |B^{*U}||B^{*V}| \le \frac{\mathrm{Vol}^2(R)}{g^2}$ and $|B^{*V}| \le |B^{*U}||B^{*V}| \le \frac{\mathrm{Vol}^2(R)}{g^2}$. Therefore,

$$|B^*| = |B^{*U}| + |B^{*V}| \le \frac{2\,\mathrm{Vol}^2(R)}{g^2}.$$

□

In practice, we are primarily interested in non-trivial subgraphs with reasonably large density values. Therefore, for the sake of presentation simplicity, we focus the following complexity analysis on the regime where $\rho^*_{\gamma,R} \ge 1$, and hence $g \ge 1$. Under this assumption, Lemma 4.4 simplifies to $|B^*| \le 2\,\mathrm{Vol}^2(R)$.

Next, we bound the degree of vertices that can appear in an optimal solution.

Lemma 4.5. *Let D^*_γ be a non-trivial optimal solution to the γ-ADSSP with $\rho^*_{\gamma,R} \ge 1$. Then for any vertex $x \in D^*_\gamma$,*

$$d_G(x) \le \mathrm{Vol}^2(R).$$

Proof. Let $D = D^*_\gamma$, and fix any vertex $x \in D$. Since D is non-trivial, we have $|D^U| \ge 1$ and $|D^V| \ge 1$.

Because $\rho_{\gamma,R}(D) \ge 1$, repeating the derivation of Lemma 3.6 for the non-strict case yields

$$\mathrm{Vol}(\overline{D}\cap R) + |E(D,\overline{D})| + \frac{|D^U|}{2\sqrt{\gamma}} + \frac{\sqrt{\gamma}\,|D^V|}{2} \le \mathrm{Vol}(R).$$

Squaring both sides yields

$$\mathrm{Vol}^2(R) \ge \left(\mathrm{Vol}(\overline{D}\cap R) + |E(D,\overline{D})| + \frac{|D^U|}{2\sqrt{\gamma}} + \frac{\sqrt{\gamma}\,|D^V|}{2}\right)^2.$$

Since all terms on the right-hand side are nonnegative, we obtain

$$\mathrm{Vol}^2(R) \ge |E(D,\overline{D})| + \left(\frac{|D^U|}{2\sqrt{\gamma}} + \frac{\sqrt{\gamma}\,|D^V|}{2}\right)^2.$$

By the AM-GM inequality,

$$\left(\frac{|D^U|}{2\sqrt{\gamma}} + \frac{\sqrt{\gamma}\,|D^V|}{2}\right)^2 \ge |D^U||D^V|.$$

Hence,

$$\mathrm{Vol}^2(R) \ge |E(D,\overline{D})| + |D^U||D^V|.$$

Now consider the fixed vertex $x \in D$. If $x \in D^U$, then $|E(x,\overline{D})| = d_G(x) - |E(x,D)|$. Since $E(x,\overline{D}) \subseteq E(D,\overline{D})$, we have

$$|E(D,\overline{D})| \ge d_G(x) - |E(x,D)|.$$

Moreover, because G is bipartite and $x \in U$, all neighbors of x that lie in D must belong to D^V. Therefore,

$$|E(x,D)| \le |D^V|.$$

Substituting the above into the previous bound yields

$$\mathrm{Vol}^2(R) \ge d_G(x) - |D^V| + |D^U||D^V| = d_G(x) + (|D^U|-1)|D^V| \ge d_G(x).$$

Similarly, if $x \in D^V$, then

$$|E(D,\overline{D})| \ge d_G(x) - |E(x,D)|$$

and, since all neighbors of x in D must lie in D^U,

$$|E(x,D)| \le |D^U|.$$

Thus,

$$\mathrm{Vol}^2(R) \ge d_G(x) - |D^U| + |D^U||D^V| = d_G(x) + |D^U|(|D^V|-1) \ge d_G(x).$$

In both cases, we obtain

$$d_G(x) \le \mathrm{Vol}^2(R).$$

□

Following Lemma 4.5, any vertex whose degree in G exceeds $\mathrm{Vol}^2(R)$ cannot belong to an optimal solution D^*_γ. Therefore, for the purpose of deriving a conservative upper bound on the local network size, such vertices may be treated as fixed on the sink side. Under this reduction, all relevant vertices in the resulting local flow network have degree bounded by $O(\mathrm{Vol}^2(R))$.

We now bound the size of the final local flow network $G^{B^*}_{\gamma,g}$.

Lemma 4.6. *The number of vertices in $G^{B^*}_{\gamma,g}$ is $O(\mathrm{Vol}^4(R))$, and the number of edges in $G^{B^*}_{\gamma,g}$ is $O(\mathrm{Vol}^4(R))$.*

Proof. The vertex set of $G^{B^*}_{\gamma,g}$ is

$$|V(G^{B^*}_{\gamma,g})| = 2 + |R| + |B^*| + |N(R \cup B^*)|.$$

The boundary size is bounded by the total degree of vertices in $R \cup B^*$ in the reduced flow network, which is at most their total degree in G. Therefore,

$$\begin{aligned}|N(R \cup B^*)| &\le \sum_{u \in R} d_G(u) + \sum_{u \in B^*} d_G(u) \\ &= \mathrm{Vol}(R) + \sum_{u \in B^*} d_G(u).\end{aligned}$$

Using $|R| \le \mathrm{Vol}(R)$, $|B^*| \le 2\mathrm{Vol}^2(R)$ from Lemma 4.4, and the fact that every relevant vertex in the reduced flow network has degree at most $\mathrm{Vol}^2(R)$, we obtain

$$\begin{aligned}|V(G^{B^*}_{\gamma,g})| &\le 2 + \mathrm{Vol}(R) + 2\mathrm{Vol}^2(R) + \mathrm{Vol}(R) + |B^*|(\mathrm{Vol}^2(R)) \\ &\le 2 + 2\mathrm{Vol}(R) + 2\mathrm{Vol}^2(R) + 2\mathrm{Vol}^2(R)\big(\mathrm{Vol}^2(R)\big) \\ &= O(\mathrm{Vol}^4(R)).\end{aligned}$$

Similarly, we bound the number of edges in $G^{B^*}_{\gamma,g}$. By construction, the edge set consists of source arcs, sink arcs, and bipartite arcs incident to vertices in $R \cup B^*$. Hence,

$$|E(G^{B^*}_{\gamma,g})| = O\left(|R| + |B^*| + |N(R \cup B^*)| + \sum_{u \in R \cup B^*} d_G(u)\right).$$

Using

$$|R| \le \mathrm{Vol}(R), \qquad |B^*| \le 2\mathrm{Vol}^2(R),$$
$$|N(R \cup B^*)| = O(\mathrm{Vol}^4(R)), \qquad \sum_{u \in R} d_G(u) = \mathrm{Vol}(R),$$

and the fact that every relevant vertex in the reduced flow network has degree at most $\mathrm{Vol}^2(R)$, we obtain

$$\sum_{u \in B^*} d_G(u) \le |B^*|(\mathrm{Vol}^2(R)) = O(\mathrm{Vol}^4(R)).$$

Therefore,

$$|E(G^{B^*}_{\gamma,g})| = O(\mathrm{Vol}^4(R)).$$

□

Theorem 4.7. *The time complexity of* LocalVerify *is* $O(\mathrm{Vol}^{10}(R))$ *using the Push-Relabel algorithm with* $O(\mathrm{Vol}^4(R))$ *space.*

Complexity of LC-ADS**.** Combining the complexity of localized verification with the parametric-search framework, we obtain the overall complexity of LC-ADS.

Theorem 4.8. *Algorithm* LC-ADS *correctly identifies the ADS* D^**, runs in* $O(n^2 \log n \cdot \mathrm{Vol}^{10}(R))$ *time, and uses* $O(n+m)$ *total space.*

Proof. **Correctness.** LC-ADS follows the same parametric-search framework as GC-ADS and differs only in the verification step. By Lemma 4.2, whenever LocalVerify terminates, the local minimum cut it returns is globally optimal. Hence, LC-ADS correctly identifies the ADS D^*.

Time Complexity. The outer loop enumerates $O(n^2)$ candidate aspect-ratio pairs. For each pair, binary search performs $O(\log n)$ threshold tests. Each threshold test invokes LocalVerify, whose worst-case complexity is $O(\mathrm{Vol}^{10}(R))$ by Theorem 4.7. Therefore, the total running time is $O(n^2 \log n \cdot \mathrm{Vol}^{10}(R))$.

Space Complexity: The total space is dominated by storing the input graph $O(n+m)$. □

Discussion. The bound $O(\mathrm{Vol}^{10}(R))$ is mainly of theoretical interest: it formalizes the locality of LocalVerify by showing that its complexity is characterized by the reference set rather than directly by the full graph size. The analysis is intentionally conservative, especially in bounding the final expansion set and the final local network. In practice, as shown by our experiments, the explored region is typically much smaller than these worst-case bounds, which helps explain the strong empirical efficiency of LC-ADS.

5 PARAMETER SEARCH OPTIMIZATION

Both GC-ADS and LC-ADS incur significant computational overhead by enumerating all $O(|U||V|)$ candidate aspect ratios γ. In this section, we develop a pruning rule that substantially reduces the number of ratios to be verified. The key idea is to exploit the relationship between the probed parameter γ and the actual aspect ratio of the optimal subgraph returned under γ, so as to eliminate dominated regions of the search space.

A convexity-based pruning technique. Recall that for any feasible subgraph D, its γ-based R-subgraph density is $\rho_{\gamma,R}(D) = \frac{2\sqrt{\gamma}\sqrt{\gamma'}}{\gamma+\gamma'}\rho_R(D)$, where $\gamma' = |D^U|/|D^V|$. For fixed D, the multiplicative factor $\frac{2\sqrt{\gamma}\sqrt{\gamma'}}{\gamma+\gamma'}$ is maximized when $\gamma = \gamma'$, and decreases as γ deviates from γ'. Intuitively, if a subgraph is evaluated under a mismatched search ratio, its γ-based density is penalized. We formalize this observation below and use it to derive a safe pruning rule.

Let $\rho^*_{\gamma,R}$ denote the optimal value for fixed γ. Let D^*_γ be an optimal solution, and let $b = |(D^*_\gamma)^U|/|(D^*_\gamma)^V|$ be its actual aspect ratio. Then, we have:

$$\rho^*_{\gamma,R} = \rho_{\gamma,R}(D^*_\gamma) = \frac{2\sqrt{\gamma}\sqrt{b}}{\gamma+b}\rho_R(D^*_\gamma). \tag{6}$$

Rearranging the above equation gives:

$$\rho_R(D^*_\gamma) = \frac{1}{2}\rho^*_{\gamma,R}\left(\sqrt{\frac{b}{\gamma}} + \sqrt{\frac{\gamma}{b}}\right). \tag{7}$$

Define $h_\gamma(x) = \sqrt{\frac{x}{\gamma}} + \sqrt{\frac{\gamma}{x}}, \quad x > 0$. Its derivative is $h'_\gamma(x) = \frac{x-\gamma}{2x\sqrt{\gamma x}}$, which shows that $h_\gamma(x)$ decreases on $(0,\gamma)$ and increases on (γ,∞). Therefore, $h_\gamma(x)$ attains its unique minimum at $x = \gamma$. This yields the following pruning rule.

Lemma 5.1. *Let* D^*_γ *be the optimal subgraph returned for parameter* γ*, and let* $b = \frac{|D^{*U}_\gamma|}{|D^{*V}_\gamma|}$ *be its actual aspect ratio. Define the conjugate point* $c = \frac{\gamma^2}{b}$*. Then for any feasible subgraph* D *whose aspect ratio* $x = \frac{|D^U|}{|D^V|}$ *lies in the interval* $[\min(b,c), \max(b,c)]$*, we have* $\rho_R(D) \le \rho_R(D^*_\gamma)$*.*

Proof. Assume for contradiction that there exists a feasible subgraph D with aspect ratio $x \in [\min(b,c), \max(b,c)]$ such that $\rho_R(D) > \rho_R(D^*_\gamma)$. By the definition $c = \gamma^2/b$, we have

$$h_\gamma(b) = \sqrt{\frac{b}{\gamma}} + \sqrt{\frac{\gamma}{b}} = \sqrt{\frac{c}{\gamma}} + \sqrt{\frac{\gamma}{c}} = h_\gamma(c).$$

Moreover, since $h_\gamma(x)$ decreases on $(0,\gamma)$ and increases on (γ,∞), and γ lies between b and c, it follows that

$$h_\gamma(x) \le h_\gamma(b) = h_\gamma(c) \quad \text{for all } x \in [\min(b,c), \max(b,c)].$$

Algorithm 3: PGC-ADS(G, R, A)

Input: Bipartite graph $G = (U, V, E)$, reference set $R \subseteq U$, anchor set $A \subseteq R$.
Output: The anchored densest subgraph D^*.

1 $\rho_R^* \leftarrow 0$; $D^* \leftarrow \emptyset$;
2 DivideConquer$(\frac{1}{|V|}, |U|)$;
3 **return** D^*;
4 **Procedure** DivideConquer(γ_l, γ_r)
5 **if** *no candidate ratio remains in* $[\gamma_l, \gamma_r]$ **then return**;
6 Choose γ_m from $[\gamma_l, \gamma_r]$ using $(\gamma_l + \gamma_r)/2$ as the probe position;
7 Execute Lines 3–14 of Algorithm 1 with parameter γ_m; let $D^*_{\gamma_m}$ be the resulting γ_m-based ADS;
8 **if** $\rho_R^* < \rho_R(D^*_{\gamma_m})$ **then** $\{\rho_R^* \leftarrow \rho_R(D^*_{\gamma_m}); D^* \leftarrow D^*_{\gamma_m};\}$
9 $b \leftarrow |{D^*_{\gamma_m}}^U|/|{D^*_{\gamma_m}}^V|$;
10 $c \leftarrow \gamma_m^2/b$;
11 **if** $b > c$ **then** Swap(b, c);
12 **if** candidate ratios remain in $[\gamma_l, b]$ **then** DivideConquer (γ_l, b);
13 **if** candidate ratios remain in $[c, \gamma_r]$ **then** DivideConquer (c, γ_r);

Therefore,

$$\rho_{\gamma,R}(D) = \frac{2}{h_\gamma(x)}\rho_R(D) \geq \frac{2}{h_\gamma(b)}\rho_R(D) > \frac{2}{h_\gamma(b)}\rho_R(D^*_\gamma) = \rho^*_{\gamma,R},$$

which contradicts the optimality of D^*_γ under parameter γ. Hence, no such subgraph D exists. □

Lemma 5.1 states that once a probe at γ returns a solution with actual ratio b, the entire interval between b and its conjugate point $c = \gamma^2/b$ can be safely pruned. Although the lemma is stated over the continuous ratio domain, the actual search is performed over the discrete candidate set $\Gamma = \left\{ \frac{i}{j} \;\middle|\; 1 \leq i \leq |U|,\ 1 \leq j \leq |V| \right\}$. Accordingly, each pruned interval removes exactly those candidate ratios in Γ that fall within it.

Example 5.2. To illustrate the pruning effect, consider a representative query instance from Exp-1 on the dblp dataset, where $|U| \approx 1.95 \times 10^6$ and $|V| \approx 5.62 \times 10^6$. The search starts from $[\frac{1}{|V|}, |U|] \approx [1.78 \times 10^{-7}, 1.95 \times 10^6]$. Suppose a probe near the midpoint returns a subgraph with $|D^U| = 5$ and $|D^V| = 1$, yielding $b = 5.0$. Since the conjugate point $c = \frac{\gamma^2}{b}$ is much larger than 5.0, Lemma 5.1 removes all candidate ratios in $\Gamma \cap [5.0, c]$, leaving only $\Gamma \cap [\frac{1}{|V|}, 5.0]$ for further exploration. A later probe at $\gamma = 2.5$ returns a subgraph with $|D^U| = 8$ and $|D^V| = 5$, yielding $b = \frac{8}{5} = 1.6$ and $c = \frac{2.5^2}{1.6} \approx 3.906$. Hence, all candidate ratios in $\Gamma \cap [1.6, 3.906]$ are pruned, and the remaining candidates are split into two subsets induced by $[\frac{1}{|V|}, 1.6]$ and $[3.906, 5.0]$.

The divide-and-conquer algorithms. Based on the above pruning rule, we design a divide-and-conquer strategy that searches only a small subset of candidate ratios in Γ. Instead of enumerating all candidates, the algorithm recursively probes one ratio from the current interval, computes the corresponding γ-based ADS, and removes the dominated interval certified by Lemma 5.1. Applying this strategy to GC-ADS and LC-ADS yields the optimized algorithms PGC-ADS and PLC-ADS, respectively. Algorithm 3 presents the pseudocode for PGC-ADS. The procedure for PLC-ADS is identical except that the global flow-based verification routine is replaced with LocalVerify.

As outlined in Algorithm 3, PGC-ADS commences by initializing the maximum R-subgraph density ρ_R^* to zero and invoking the recursive divide-and-conquer strategy over the maximal theoretical interval $[\frac{1}{|V|}, |U|]$ (Lines 1–2). Within each recursive step, the algorithm first checks whether the current interval still contains candidate ratios; otherwise, the recursion terminates (Line 5). For a valid interval, it selects a probe point γ_m from the current search range, using the midpoint $(\gamma_l + \gamma_r)/2$ as the guiding position (Line 6), and executes the core verification procedure based on network flow to obtain the corresponding γ_m-based ADS $D^*_{\gamma_m}$ (Line 7). If the R-subgraph density of $D^*_{\gamma_m}$ exceeds the current global optimum, the best solution is updated immediately (Line 8). Subsequently, the algorithm derives the actual aspect ratio b and its conjugate $c = \gamma_m^2/b$ (Lines 9–10); to facilitate consistent interval processing, it enforces $b \leq c$ by swapping the values if necessary (Line 11). Finally, leveraging the convexity-based pruning rule, the algorithm prunes the dominated middle interval between b and c, and recursively explores the two remaining subranges only if they still contain candidate ratios (Lines 12–13). Upon completion of the recursive search, the algorithm returns the ADS D^* (Line 3).

Complexity analysis. We next analyze the correctness and complexity of PGC-ADS and PLC-ADS.

Theorem 5.3. *Algorithms* PGC-ADS *and* PLC-ADS *correctly identify the optimal ADS D^*. Their time complexities are $O(k \cdot n^2\sqrt{m}\log n)$ and $O(k \cdot \mathrm{Vol}^{10}(R)\log n)$, respectively, where k is the number of probed candidate ratios, $n = |U| + |V|$, and $m = |E|$. Both algorithms maintain a space complexity of $O(n + m)$.*

Discussion. The efficiency of PGC-ADS and PLC-ADS depends on the number of probed candidate ratios, denoted by k. In the worst case, the pruning interval certified by Lemma 5.1 may contain only a small number of feasible candidates from Γ, so the recursive procedure may examine a large fraction of the candidate set. Since $|\Gamma| = O(|U||V|)$, the worst-case asymptotic complexity matches that of exhaustive enumeration. In practice, however, a single probe often removes a substantial dominated interval, thereby eliminating many candidate ratios at once. As a result, the empirical value of k is typically much smaller than $|\Gamma|$, leading to a significant speedup over the baseline algorithms. The space complexity remains $O(n+m)$, as it is dominated by the graph representation and the corresponding flow network.

6 EXPERIMENTS

6.1 Experiment settings

Different algorithms. We evaluate GC-ADS and LC-ADS, and their optimized variants PGC-ADS and PLC-ADS. All implementations employ Push–Relabel to solve the underlying max-flow subproblems. Since no prior exact method exists for anchored densest subgraph search in bipartite graphs, our evaluation focuses on a component-wise comparison among these variants.

Datasets and environment. We use 16 real-world bipartite graphs from the Network Repository (https://networkrepository.com/) and the Koblenz Network Collection (KONECT) (http://konect.cc/), spanning diverse domains including social, interaction, hyperlink, edit, collaboration, and co-purchasing networks. All graphs are treated as undirected and unweighted. Consistent with our problem definition, we designate the smaller side of each bipartite graph as U (i.e., $|U| \leq |V|$), from which both A and R are selected. All algorithms are implemented in C++ and compiled with GCC using the `-O3` optimization flag. Experiments are conducted on a Linux server

Table 1: Graph datasets statistics

Name	Abbr.	Type	$\|U\|$	$\|V\|$	$\|E\|$
wikibooksie	WB	Interaction	137	558	651
moreno	MO	Social	551	829	1,476
wikieditdv	WD	Edit	195	861	2,079
fjwiki	FW	Hyperlink	612	1,898	5,959
escorts	ES	Interaction	6,624	10,106	39,044
marvel	MA	Social	6,486	12,942	96,662
youtube	YT	Social	30,087	94,238	293,360
book	BK	Social	77,802	185,955	433,652
git	GT	Social	56,519	120,867	440,237
stackoverflow	SO	Interaction	96,678	545,195	1,301,942
imdb	IM	Collaboration	186,414	685,568	2,715,604
amazon	AM	Co-purchasing	1,230,915	2,146,057	5,743,258
dblp	DB	Collaboration	1,953,085	5,624,219	12,282,059
pokec	PO	Social	1,151,281	1,632,794	22,301,964
livejournal	LJ	Social	3,201,203	7,489,073	112,307,385
orkut	OR	Social	2,783,196	8,730,857	327,037,487

with an AMD 3995WX 64-core 2.2 GHz CPU and 256 GB RAM. The timeout is set to 1,000 seconds per query.

Query generation. For each dataset, we generate query instances by constructing the anchor set A and the reference set R using a bipartite adaptation of the query-generation strategy for anchored densest subgraph search in general graphs proposed in [12].

We first randomly sample a vertex $u \in U$ as the initial seed. To construct the anchor set A, we consider all vertices in U that are reachable from u within two hops in the bipartite graph. For each such vertex v, we compute the Jaccard similarity between the neighbor sets of u and v. We then select the top-k vertices with the highest similarity scores (default: $k = 3$) and form the anchor set A by combining these vertices with the initial seed u, resulting in $|A| = 4$ by default.

Given the anchor set A, we construct the reference set R by performing random walks starting from vertices in A. Specifically, for each vertex in A, we conduct 3 independent random walks of length 8, where each step uniformly samples one neighboring vertex. To avoid introducing high-degree vertices during this process, we apply a degree-based cap when adding visited vertices to R. Let $d_{\max} = \max_{u \in A} d(u)$ denote the maximum degree among the anchor vertices. We define a scaling factor d_c as 10 if $d_{\max} < |V|/e^{10}$; otherwise, $d_c = \left\lceil \log\left(\frac{|V|}{d_{\max}}\right) \right\rceil$. The degree threshold is then set to $\tau = d_c \cdot d_{\max}$. During the random walks, a visited vertex v is added to R if $v \in A$ or $d(v) \leq \tau$. This guarantees that $A \subseteq R$.

For each dataset, we generate 100 query instances. To focus on queries with meaningful locality, we apply a simple locality filter during query generation: if the explored region contains more than half of the edges of the original graph, the query is regarded as non-local and discarded. We repeat the generation process until 100 valid queries are obtained for each dataset. The same query set is used for all compared algorithms on each dataset.

6.2 Performance studies

Exp-1: The runtime of different algorithms. Figure 5 reports the average running time of all algorithms over 100 random queries on each dataset. On the small datasets (WB, MO, and WD), all four algorithms finish within the time limit, enabling a full comparison. The effect of pruning is clear: PGC-ADS and PLC-ADS substantially outperform GC-ADS and LC-ADS by avoiding exhaustive enumeration over γ. For example, on MO, PGC-ADS reduces the runtime from 3,025 seconds to 0.06 seconds, a speedup of over 50,000×. The benefit of local verification is also consistent: LC-ADS is faster than GC-ADS (e.g., by 7.9× on MO), and this advantage remains after optimization, with PLC-ADS being about 6.7× faster than PGC-ADS.

As the graph size increases (from FW to PO), the algorithms GC-ADS and LC-ADS no longer finish within the time limit because exhaustive γ enumeration becomes prohibitive. In contrast, the optimized variants remain efficient, and the gap between the local and global strategies widens. On PO, for example, PGC-ADS and PLC-ADS take 2,589 seconds and 24 seconds on average, respectively, so PLC-ADS is 108× faster. This result shows the importance of both pruning and local verification on medium-sized graphs.

On the largest datasets (LJ and OR), PGC-ADS fails to return results (reported as INF) because constructing the global flow network becomes infeasible at this scale and leads to out-of-memory (OOM) errors. In contrast, PLC-ADS still finishes on LJ and OR in 3,000 seconds. These results further demonstrate the better scalability of the local strategy on massive graphs.

Exp-2: Memory usage comparison. Figure 6 reports the peak memory usage of all algorithms. In the global category, GC-ADS and PGC-ADS construct the same flow network over the entire graph and therefore have the same memory footprint. In the local category, memory is measured as the peak over all local subgraphs constructed for different γ values. Since LC-ADS exhaustively enumerates γ while PLC-ADS prunes invalid candidates, PLC-ADS explores only a subset of the subgraphs examined by LC-ADS. As a result, PLC-ADS consistently uses no more memory than LC-ADS.

We next compare the optimized variants, PGC-ADS and PLC-ADS. PLC-ADS is substantially more memory-efficient, often by orders of magnitude on medium-sized datasets. For example, on DB, PLC-ADS uses only 0.003 GB, compared with 3.14 GB for PGC-ADS, a reduction of over 1000×. This smaller flow network also helps explain the runtime gap in Exp-1. On the largest datasets (LJ and OR), PGC-ADS fails due to out-of-memory (OOM), whereas PLC-ADS remains feasible; even on OR, its peak memory usage is only 18.6 GB, well within the memory budget.

Exp-3: Pruning effectiveness. Figure 7 reports the average number of enumerated γ values. Since this number depends only on the pruning strategy, GC-ADS and LC-ADS have the same enumeration count, and so do PGC-ADS and PLC-ADS. We therefore use LC-ADS and PLC-ADS as representatives.

Without pruning, LC-ADS enumerates an extremely large number of candidates, reaching 1.47×10^{13} on OR. In contrast, PLC-ADS enumerates fewer than 10 candidates on average for most datasets (e.g., 6.23 on OR), reducing the search space by more than 12 orders of magnitude. This result is consistent with the structure of anchored densest subgraphs: meaningful solutions usually correspond to reasonably balanced bipartite partitions, whereas highly skewed γ values are unlikely to be optimal. Our pruning rule exploits this property to discard large ranges of unlikely γ values and focus on a much smaller candidate set.

This reduction in γ enumeration is a main reason for the runtime gains in Exp-1. By avoiding a large number of unnecessary max-flow computations, both PGC-ADS and PLC-ADS can answer queries much more efficiently.

Exp-4: Verifying locality. To better understand the efficiency gain sobserved in Exp-1, we examine the relationship between speedup and locality. Figure 8 plots, for each local–global pair, the speedup of the local algorithm over its global counterpart against the *edge exploration ratio*, i.e., the ratio between the average number of edges in the largest local subgraph constructed per query and the total number of edges in G. Datasets WB, MO, and

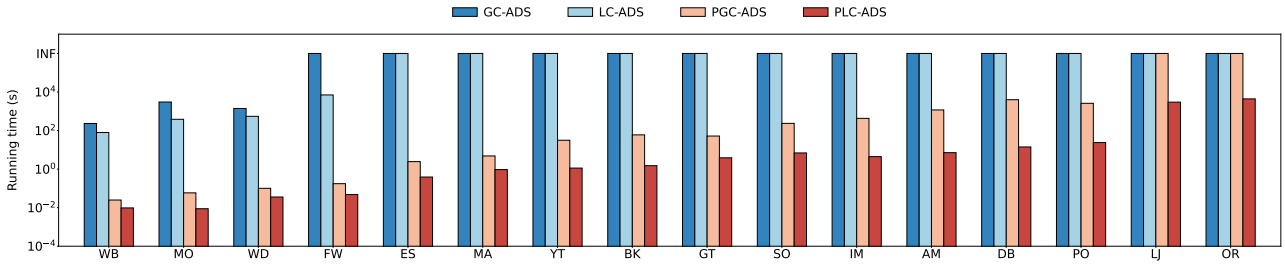


Figure 5: Comparison of running time.

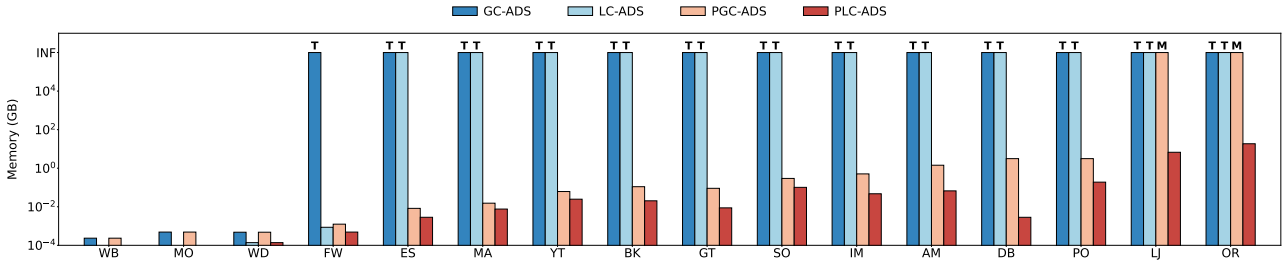


Figure 6: Comparison of memory cost. "T" denotes timeout and "M" denotes out-of-memory.

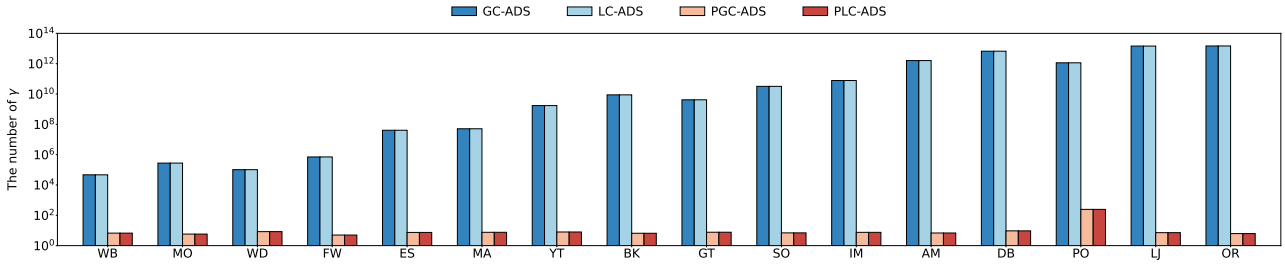


Figure 7: The average number of enumerated γ values by different algorithms.

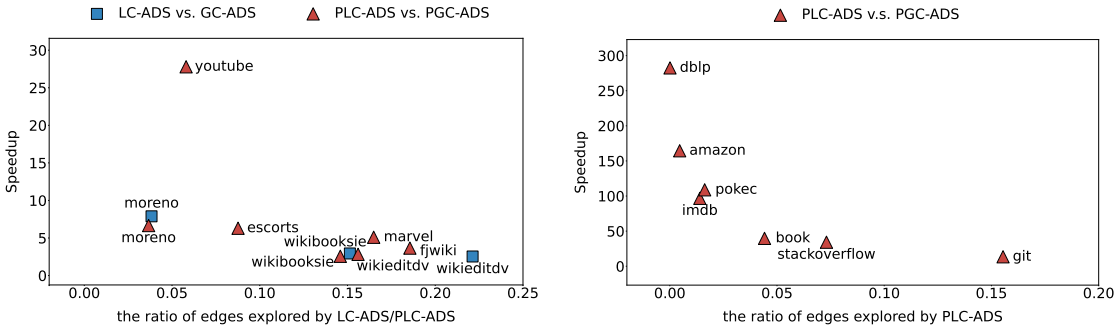


Figure 8: Speedup vs. edge exploration ratio. Blue squares denote LC-ADS/GC-ADS **and are shown only where** GC-ADS **finishes; red triangles denote** PLC-ADS/PGC-ADS **and are shown only where** PGC-ADS **finishes.**

WD each contribute two points because both (LC-ADS, GC-ADS) and (PLC-ADS, PGC-ADS) complete on them; for the remaining datasets, only (PLC-ADS, PGC-ADS) is reported. Datasets LJ and OR are omitted because PGC-ADS does not finish.

As shown in Figure 8, the edge exploration ratio is generally small, indicating that both LC-ADS and PLC-ADS typically confine computation to a compact neighborhood around the reference vertices. For example, in the left panel, PLC-ADS achieves a 27× speedup on youtube with an exploration ratio well below 0.1. The right panel shows a clear trend: smaller exploration ratios generally translate into larger speedups. On dblp, the local subgraphs processed by PLC-ADS contain less than 1% of all edges on average, yielding a speedup of nearly 280×. Similar behavior is observed on amazon and pokec, where exploration ratios below 0.03 correspond to speedups exceeding 100×. In contrast, on GT, the exploration ratio is higher (about 0.16), and the resulting speedup is correspondingly smaller (about 15×). These results show that local verification gains efficiency by avoiding irrelevant parts of the graph.

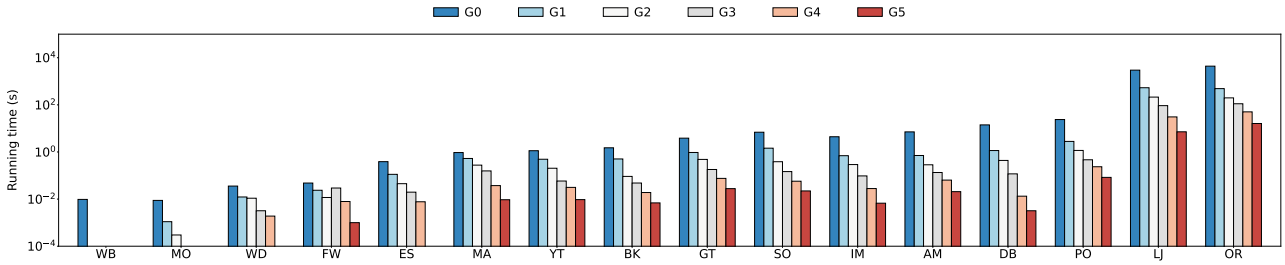


Figure 9: The running time of PLC-ADS **with different graph density.**

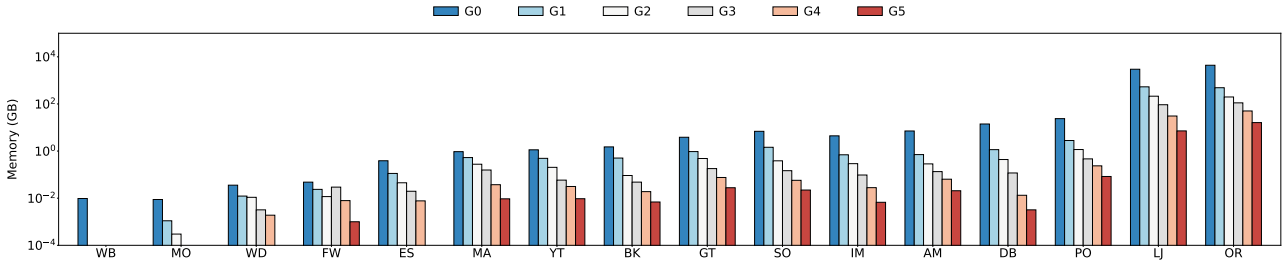


Figure 10: The memory cost of PLC-ADS **with different graph density.**

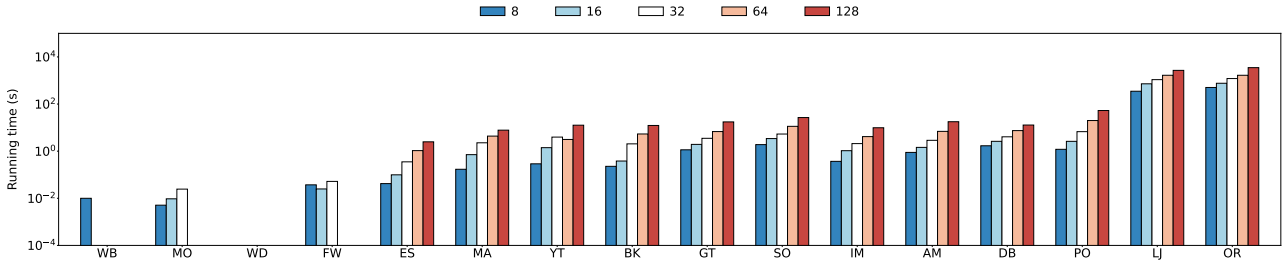


Figure 11: The running time of PLC-ADS **with different** $|R|$**.**

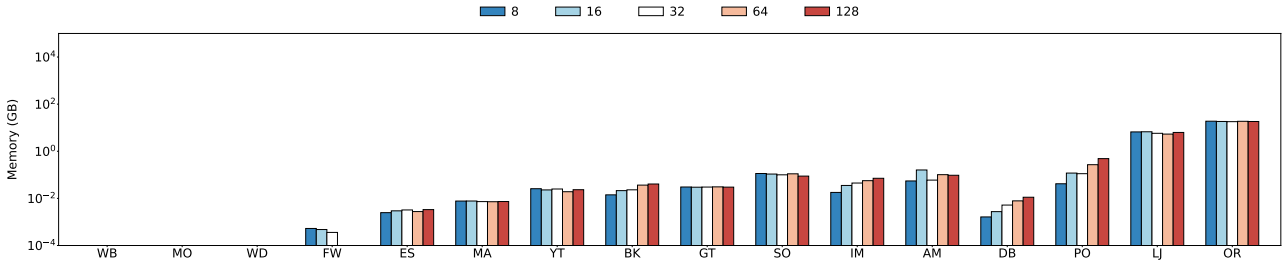
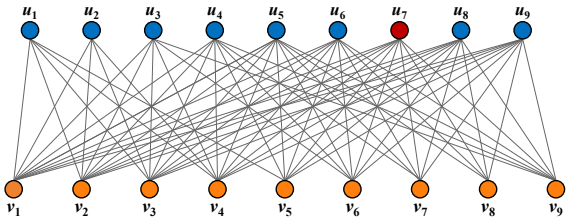


Figure 12: The memory cost of PLC-ADS **with different** $|R|$**.**

Exp-5: Sensitivity of PLC-ADS **to graph density.** We study how graph density affects PLC-ADS by constructing a sequence of progressively sparser subgraphs. Starting from the original graph G_0, we generate $G_1, \ldots, G_5$ by retaining each edge independently with probability 0.5^i and then extracting the largest connected component. Thus, larger i yields a smaller and sparser graph. For each G_i, we generate 1,000 queries using the default settings. We exclude any G_i with fewer than 128 vertices. Figure 9 and Figure 10 report the average running time and peak memory usage of PLC-ADS, respectively. Both runtime and memory decrease monotonically as i increases, i.e., as the graph becomes sparser. On large datasets such as LJ and OR, the gap between G_0 and G_5 spans several orders of magnitude. This behavior is expected because PLC-ADS expands a query-dependent local subgraph, and the cost of this expansion is closely tied to local density. In sparser graphs, fewer edges are explored, so the local subgraphs are smaller, which reduces both memory usage and max-flow time. These results show that PLC-ADS adapts well to graph density: it is expensive only when the local structure itself is dense.

Exp-6: Sensitivity of PLC-ADS **on reference set size.** We next study the effect of the reference set size $r = |R|$ on PLC-ADS. We vary r in $\{8, 16, 32, 64, 128\}$. Following the default query generation method, we set $|A| = r/4$ for each r to keep the query setting consistent. For each value of r, we run 100 queries on every dataset. Figure 11 and Figure 12 report the average running time



Author IDs: u_1 Audrey Der; u_2 Vivian Lai; u_3 Xin Dai; u_4 Liang Wang; u_5 Chin-Chia M. Yeh; u_6 Wei Zhang; u_7 Junpeng Wang; u_8 Zhongfang Zhuang; u_9 Yan Zheng.

Paper IDs: v_1 Toward a Foundation Model for Time Series Data; v_2 An Efficient Content-based Time Series Retrieval System; v_3 A Systematic Evaluation of Generated Time Series...; v_4 Matrix Profile for Anomaly Detection on Multidimensional Time Series; v_5 PUPAE: Intuitive and Actionable Explanations for Time Series Anomalies; v_6 FATA-Trans: Field And Time-Aware Transformer for Sequential Tabular Data; v_7 Online Multi-horizon Transaction Metric Estimation...; v_8 Error-bounded Approximate Time Series Joins using Compact Dictionary...; v_9 Spatial-Temporal Graph Boosting Networks...

Figure 13: ADS result on dblp-DBDM**.**

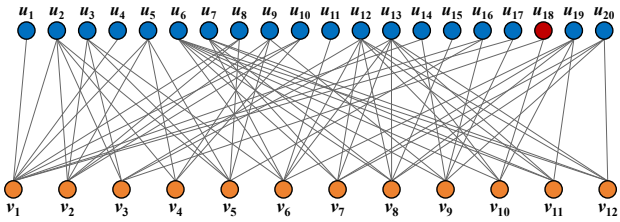
and peak memory usage, respectively. As r increases, both runtime and memory grow moderately. This trend is expected because a larger reference set imposes stronger connectivity constraints, forcing the algorithm to explore a broader local neighborhood to connect all anchors, which increases both computational and storage cost. Importantly, the growth is generally close to linear rather than exponential. Even at $r = 128$, which represents a demanding query setting, PLC-ADS remains stable. This result indicates that PLC-ADS scales well with the size of the reference set.

6.3 Case studies

We present four qualitative case studies: ADS results on dblp-DBDM, amazon-BOOK, and imdb-YEAR, followed by a comparison with alternative community search methods. Case-study queries are generated using the protocol in Section 6.1. Specifically, we choose a seed vertex x uniformly at random from one partition, set $A = \{x\}$, and generate the reference set R by launching 15 random walks from x, each with maximum length 16. In all visualizations, the red vertex denotes the anchor; blue vertices belong to the reference set R; green vertices are on the same side as A and R but outside R; and orange vertices lie on the other side of the bipartite graph.

ADS case study on dblp-DBDM**.** We construct dblp-DBDM, an author–paper bipartite graph, from DBLP[1] records in major database and data mining venues such as SIGMOD and VLDB. The resulting graph contains 61,652 author vertices, 44,375 paper vertices, and 175,433 edges. In this case study, the anchor is placed on the *author side* so that the query targets a coherent group of collaborators, while R captures their broader academic neighborhood. Figure 13 shows a representative ADS instance. Structurally, the result reveals a compact collaboration group (u_1–u_9). These authors are not globally dominant in the full DBLP graph, but they form a dense local community around the query. Semantically, the extracted papers (v_1–v_9) are strongly centered on *time series analysis*, covering topics such as foundation models, retrieval, anomaly detection, and spatio-temporal learning. This example shows that ADS can recover a structurally dense and topically coherent local research community.

[1]https://dblp.uni-trier.de/xml/



Person IDs: u_1 Alejandro Goic; u_2 Antonia Zegers; u_3 Marcial Tagle; u_4 Francisco Reyes; u_5 Alfredo Castro; u_6 Ramón Llao; u_7 Nicolás Martínez; u_8 Jaime Vadell; u_9 Amparo Noguera; u_{10} Marcelo Alonso; u_{11} Andrea Osvárt; u_{12} Ariel Levy; u_{13} Paz Bascuñán; u_{14} Alejandro Sieveking; u_{15} Julio Jung; u_{16} Diego Muñoz; u_{17} Roberto Farías; u_{18} Carolina Paulsen; u_{19} Ignacia Allamand; u_{20} Claudia Celedón.

Movie IDs: v_1 The Club (2015); v_2 Post Mortem (2010); v_3 Qué pena tu boda (2011); v_4 Life Kills Me (2007); v_5 No Filter (2016); v_6 Best Worst Friends (2013); v_7 Qué pena tu familia (2012); v_8 Aftershock (2012); v_9 No estoy loca (2018); v_{10} Tony Manero (2008); v_{11} Qué pena tu vida (2010); v_{12} El Conde (2023).

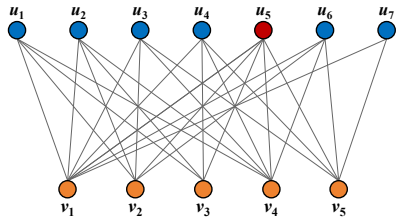
Figure 14: ADS result on imdb-YEAR**.**

ADS case study on imdb-YEAR**.** We construct imdb-YEAR, a people–movie bipartite graph, from the IMDB[2] dataset by retaining feature films released between 2000 and 2024 and restricting person nodes to primary roles, including actors, actresses, and directors. The resulting graph contains 23,713 person vertices, 55,260 movie vertices, and 266,631 edges. In this case study, the anchor is placed on the *person side* so that the query targets a coherent group of filmmakers, while R captures their extended collaboration context. Figure 14 shows a representative ADS instance. Structurally, the result reveals a repertory-company-style collaboration pattern: actors such as Alfredo Castro (u_5) and Antonia Zegers (u_2) repeatedly co-occur across multiple films. These actors are not globally dominant in the full IMDB graph, where global dense subgraphs are typically dominated by large mainstream productions. ADS instead identifies a mid-scale but highly coherent local community consistent with both the anchor set A and the reference set R. Semantically, the extracted movies (v_1–v_{12}) form a clear regional cluster of Chilean cinema, including *The Club*, *Tony Manero*, and *El Conde*, with shared themes of social realism and political critique.

ADS case study on amazon-BOOK**.** We construct amazon-BOOK, a user–book bipartite graph, from the Amazon Books[3] dataset by retaining only reviews with ratings 4 or 5 so that edges represent strong positive preferences. The resulting graph contains 703,533 user vertices, 1,849,283 book vertices, and 22,787,122 edges. In this case study, the anchor is placed on the *book side* so that the query targets a coherent reading niche, while R captures the surrounding reading interests. Figure 15 shows a representative ADS instance. Structurally, the result reveals a compact group of users connected through a dense local interest pattern. These users are not necessarily the most active reviewers in the full Amazon graph, where global dense subgraphs are more likely to be dominated by highly popular books and broadly connected users. ADS instead identifies a small but highly coherent local community consistent with both the anchor set A and the reference set R. Semantically, the extracted books all belong to the *Lima Bear Stories* series, including *The Labyrinth* and *Bully Bean*. This result shows that ADS can recover a structurally dense and topically coherent niche community even in a very large interaction graph.

Comparison with alternative community search methods. As the first formulation of anchored densest subgraph search for

[2]https://developer.imdb.com/non-commercial-datasets/
[3]https://nijianmo.github.io/amazon/index.html



Book IDs: u_1 The Labyrinth (Lima Bear Stories); u_2 Bully Bean (Lima Bear Stories); u_3 The Megasaurus (Lima Bear Stories); u_4 The Cave Monster; u_5 How Back-Back Got His Name (Lima Bear Stories); u_6 The Cave Monster (Lima Bear Stories); u_7 The Treasure Hunt (Lima Bear Stories).

Figure 15: ADS result on amazon-BOOK**.**

bipartite graphs, ADS has no direct baseline. We therefore compare it with representative community search methods adapted to our setting, including methods originally designed for general graphs—Personalized PageRank (PPR) [3], Memory Random Walk (MRW) [6], and Unipartite ADS [12]—as well as a representative cohesive subgraph model for bipartite graphs, namely the α-β core [13]. Figure 16 illustrates the results on imdb-YEAR, while the discussion below summarizes the behavior of these methods across all three datasets.

For PPR, we use push-based approximate PPR ($\alpha = 0.15$, $\varepsilon = 10^{-4}$) initialized uniformly over R. For MRW, we use the standard single-walker setting [6] ($\alpha = 0.1$, $\beta = 0.6$, $K = 5$) restarting from the anchor. In both cases, vertices are ranked by convergence scores (degree-normalized for PPR), and a sweep cut is applied to select the prefix minimizing conductance. Across datasets, both methods tend to return broader and less focused communities than ADS, especially on imdb-YEAR and dblp-DBDM. As shown in Figure 16(a) and Figure 16(b), on imdb-YEAR, PPR produces a subgraph with 79 left vertices, 239 right vertices, and 580 edges, while MRW returns a smaller but still loose result with 27 left vertices, 30 right vertices, and 75 edges. In contrast, ADS (Figure 14) extracts a much more compact community with 20 left vertices, 12 right vertices, and 71 edges; this result is a subgraph of both the PPR and MRW outputs. This difference is consistent with our objective, which explicitly penalizes vertices outside R and thus discourages the over-expansion common in conductance-based local clustering.

When Unipartite ADS is applied directly to the bipartite graph, it optimizes the generic R-subgraph density $w(D)/(|D^U| + |D^V|)$ rather than the bipartite-aware objective $w(D)/\sqrt{|D^U||D^V|}$, where $w(D) = 2|E(D)| - \sum_{x \in D \setminus R} d_G(x)$. We found that on dblp-DBDM and amazon-BOOK, it returns the same communities as ADS. This indicates that, for these two instances, the difference between additive and geometric normalization does not affect the identity of the optimal local community: although the two objectives are defined differently, they happen to rank the same subgraph highest. In contrast, on imdb-YEAR, as shown in Figure 16(c), Unipartite ADS yields a larger but less focused result, with 21 left vertices, 20 right vertices, and 97 edges. This is consistent with the inherently asymmetric structure of filmmaker–movie collaborations, where a relatively large set of people may connect densely to a smaller set of shared movies. The bipartite-aware objective preserves such imbalance, whereas the unipartite objective tends to favor a more balanced but looser expansion.

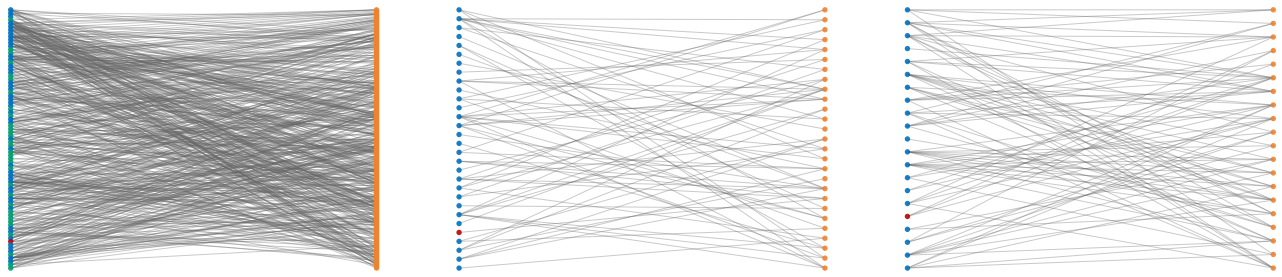
We also compare against the global cohesive baseline given by the α-β core. We extract the maximal core containing the anchor under two settings: one maximizing $\alpha \times \beta$, and one maximizing α under the symmetric constraint $\alpha = \beta$. These query-independent cores are generally too large for interpretable local analysis. On imdb-YEAR, the maximal asymmetric core ($\alpha = 3$, $\beta = 5$) contains 187,375 edges, and the maximal symmetric core ($\alpha = \beta = 3$) contains 246,697 edges. On amazon-BOOK, the corresponding numbers are 445,465 and 18,839,442 edges. Even on dblp-DBDM, where the maximal asymmetric core ($\alpha = 6$, $\beta = 8$) is a small 48-edge biclique, the symmetric core ($\alpha = \beta = 6$) still expands to 143 edges. These results show that query-independent global cores are not well suited to anchored local community discovery.

7 RELATED WORK

Densest Subgraph Discovery on General Graphs. Densest Subgraph Discovery (DSD) is a fundamental primitive in graph mining [8, 15, 17, 24, 33], typically aiming to maximize the average degree $|E(S)|/|V(S)|$. Classic algorithms evolved from Goldberg's exact max-flow solution [17] to Charikar's scalable greedy peeling (1/2-approximation) [8], later optimized for distributed [4] and streaming settings [14]. Beyond edge density, research has expanded to structural metrics like cliques [15, 37, 42, 47] and constraint-driven variants to support customized exploration. These include size-bounded discovery [5, 38], community search with query vertices [40, 45], and optimization using attractors or penalties [12, 16]. Crucially, all these approaches are strictly tailored for simple unipartite graphs. Directly applying these methods to bipartite graphs is inadequate, as they ignore the heterogeneous semantics of the two partitions and fail to properly capture the cross-partition structure inherent in bipartite networks.

Densest Subgraph Discovery on Bipartite Graphs. Given a bipartite graph $G = (U, V, E)$, DSD seeks to identify subsets $S \subseteq U, T \subseteq V$ maximizing the density $\rho(S, T) = |E(S, T)|/\sqrt{|S||T|}$ [8, 41, 43, 56, 57]. Methodologically, this parallels directed DSD, where finding dense source/target sets is mathematically equivalent [8, 21, 41]. Leveraging this, foundational frameworks by Charikar [8] and Khuller and Saha [22] established both exact solutions (via LP/Max-Flow) and efficient approximations (via peeling). Recent advances focus on scalability and advanced optimization: Bahmani et al. [4] adapted peeling for streaming; Ma et al. introduced convex programming [35] and core-based decomposition [36] to enable efficient exact and approximate computation; and Luo et al. [34] developed parallel algorithms for massive graphs. However, these approaches are inherently global and query-agnostic. They optimize a holistic objective, inevitably identifying the single densest core of the entire network (e.g., global hubs), and consequently lack mechanisms to support anchors for local, query-relevant community discovery.

Community Search on Bipartite Graphs. Research in this domain is fundamentally built upon cohesive subgraph models. While early works utilized strict bicliques [1, 9], quasi-biclique [25, 44], k-biplexes [11, 52, 53], the degree-based (α, β)-core [13, 31, 32] has become the most prevalent model due to its linear-time efficiency, alongside higher-order butterfly-based bitrusses [43, 48, 49, 63]. Based on these structural foundations, existing literature falls into two paradigms. Query-dependent search identifies local structures (typically (α, β)-cores) containing a query vertex q, often incorporating size bounds [26, 59, 61] or attribute constraints [20, 51, 60] to refine relevance. Conversely, query-independent search detects global communities without user-specified vertices, such as top-r influential communities [55], pareto-optimal sets [58], or temporal communities [27, 28]. However, regardless of the paradigm, these methods are primarily built on structural constraints rather than



(a) The subgraph obtained by PPR (b) The subgraph obtained by MRW (c) The subgraph obtained by Unipartite ADS

Figure 16: Comparison with alternative community search methods on imdb-YEAR**. The left and right partitions correspond to the anchor and non-anchor sides, respectively.**

density-maximization objectives, making them less suitable for anchor-guided discovery based on density optimization.

8 CONCLUSION

In this paper, we formulate the anchored densest subgraph search problem on bipartite graphs. We first develop a baseline algorithm, GC-ADS, based on a parametric maximum-flow framework over the entire graph. This baseline reveals two main challenges: the high cost of global flow computation and the large search space of the ratio parameter γ. To address the first challenge, we introduce a localized verification strategy, leading to LC-ADS. To address the second, we develop a convexity-based pruning strategy that substantially reduces the number of γ values to be examined. Combining this pruning strategy with GC-ADS and LC-ADS yields the optimized algorithms PGC-ADS and PLC-ADS, respectively. Experiments on real datasets show that these optimizations lead to substantial efficiency gains, with PLC-ADS achieving orders-of-magnitude speedups while maintaining scalability on large graphs. Case studies further show that the extracted subgraphs are compact, interpretable, and semantically coherent.

REFERENCES

[1] Gabriela Alexe, Sorin Alexe, Yves Crama, Stephan Foldes, Peter L. Hammer, and Bruno Simeone. 2004. Consensus algorithms for the generation of all maximal bicliques. *Discret. Appl. Math.* 145, 1 (2004), 11–21.

[2] Sihem Amer-Yahia, Senjuti Basu Roy, Ashish Chawla, Gautam Das, and Cong Yu. 2009. Group Recommendation: Semantics and Efficiency. *Proc. VLDB Endow.* 2, 1 (2009), 754–765.

[3] Reid Andersen, Fan Chung, and Kevin Lang. 2006. Local graph partitioning using pagerank vectors. In *FOCS*. 475–486.

[4] Bahman Bahmani, Ravi Kumar, and Sergei Vassilvitskii. 2012. Densest Subgraph in Streaming and MapReduce. *Proc. VLDB Endow.* 5, 5 (2012), 454–465.

[5] Aditya Bhaskara, Moses Charikar, Eden Chlamtac, Uriel Feige, and Aravindan Vijayaraghavan. 2010. Detecting high log-densities: an $O(n^{1/4})$ approximation for densest k-subgraph. In *STOC*. 201–210.

[6] Yuchen Bian, Yaowei Yan, Wei Cheng, Wei Wang, Dongsheng Luo, and Xiang Zhang. 2018. On multi-query local community detection. In *ICDM*. 9–18.

[7] Lucas Augusto Montalvão Costa Carvalho and Hendrik Teixeira Macedo. 2013. Users' satisfaction in recommendation systems for groups: an approach based on noncooperative games. In *WWW*. 951–958.

[8] Moses Charikar. 2000. Greedy approximation algorithms for finding dense components in a graph. In *APPROX (Lecture Notes in Computer Science, Vol. 1913)*. 84–95.

[9] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2022. Efficient Maximal Biclique Enumeration for Large Sparse Bipartite Graphs. *Proc. VLDB Endow.* 15, 8 (2022), 1559–1571.

[10] Zi Chen, Yiwei Zhao, Long Yuan, Xuemin Lin, and Kai Wang. 2023. Index-Based Biclique Percolation Communities Search on Bipartite Graphs. In *ICDE*. 2699–2712.

[11] Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024. Efficient Maximal Biplex Enumerations with Improved Worst-Case Time Guarantee. *Proc. ACM Manag. Data* 2, 3 (2024), 135.

[12] Yizhou Dai, Miao Qiao, and Lijun Chang. 2022. Anchored Densest Subgraph. In *SIGMOD*. 1200–1213.

[13] Danhao Ding, Hui Li, Zhipeng Huang, and Nikos Mamoulis. 2017. Efficient Fault-Tolerant Group Recommendation Using alpha-beta-core. In *CIKM*. 2047–2050.

[14] Alessandro Epasto, Silvio Lattanzi, and Mauro Sozio. 2015. Efficient Densest Subgraph Computation in Evolving Graphs. In *WWW*. 300–310.

[15] Yixiang Fang, Kaiqiang Yu, Reynold Cheng, Laks V. S. Lakshmanan, and Xuemin Lin. 2019. Efficient Algorithms for Densest Subgraph Discovery. *Proc. VLDB Endow.* 12, 11 (2019), 1719–1732.

[16] Adriano Fazzone, Tommaso Lanciano, Riccardo Denni, Charalampos E. Tsourakakis, and Francesco Bonchi. 2022. Discovering Polarization Niches via Dense Subgraphs with Attractors and Repulsers. *Proc. VLDB Endow.* 15, 13 (2022), 3883–3896.

[17] Andrew V Goldberg. 1984. Finding a maximum density subgraph. (1984).

[18] Jagadeesh Gorla, Neal Lathia, Stephen Robertson, and Jun Wang. 2013. Probabilistic group recommendation via information matching. In *WWW*. 495–504.

[19] Jelena Grujić. 2008. Movies recommendation networks as bipartite graphs. *Int. J. Mod. Phys. C* 19, 11 (2008), 1679–1688.

[20] Fangda Guo, Xuanpu Luo, Yanghao Liu, Guoxin Chen, Yongqing Wang, Huawei Shen, and Xueqi Cheng. 2024. ESC: Edge-attributed Skyline Community Search in Large-scale Bipartite Graphs. *CoRR* abs/2401.12895 (2024).

[21] Ravindran Kannan and V Vinay. 1999. *Analyzing the structure of large graphs.* Universität Bonn. Institut für Ökonometrie und Operations Research.

[22] Samir Khuller and Barna Saha. 2009. On Finding Dense Subgraphs. In *ICALP (Lecture Notes in Computer Science, Vol. 5555)*. Springer, 597–608.

[23] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *IEEE Comput.* 42, 8 (2009), 30–37.

[24] Victor E. Lee, Ning Ruan, Ruoming Jin, and Charu C. Aggarwal. 2010. A Survey of Algorithms for Dense Subgraph Discovery. In *Managing and Mining Graph Data*. Advances in Database Systems, Vol. 40. 303–336.

[25] Jinyan Li, Kelvin Sim, Guimei Liu, and Limsoon Wong. 2008. Maximal Quasi-Bicliques with Balanced Noise Tolerance: Concepts and Co-clustering Applications. In *SDM*. 72–83.

[26] Mo Li, Renata Borovica-Gajic, Farhana Murtaza Choudhury, Ningning Cui, and Linlin Ding. 2024. Maximal size constraint community search over bipartite graphs. *Knowl.-Based Syst.* 297 (2024), 111961.

[27] Mo Li, Zhiran Xie, and Linlin Ding. 2023. Persistent Community Search Over Temporal Bipartite Graphs. In *ADMA*. 324–339.

[28] Mo Li, Zhiran Xie, and Linlin Ding. 2024. Reliable Community Search over Dynamic Bipartite Graphs. In *WISA*. 298–307.

[29] Shunyang Li, Kai Wang, Xuemin Lin, Wenjie Zhang, Yizhang He, and Long Yuan. 2024. Querying Historical Cohesive Subgraphs Over Temporal Bipartite Graphs. In *ICDE*. 2503–2516.

[30] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Comput.* 7, 1 (2003), 76–80.

[31] Boge Liu, Long Yuan, Xuemin Lin, Lu Qin, Wenjie Zhang, and Jingren Zhou. 2019. Efficient (a,β)-core Computation: an Index-based Approach. In *WWW*. 1130–1141.

[32] Boge Liu, Long Yuan, Xuemin Lin, Lu Qin, Wenjie Zhang, and Jingren Zhou. 2020. Efficient (α, β)-core computation in bipartite graphs. *VLDB J.* 29, 5 (2020), 1075–1099.

[33] Wensheng Luo, Chenhao Ma, Yixiang Fang, and Laks VS Lakshmanan. 2023. A survey of densest subgraph discovery on large graphs. *arXiv preprint arXiv:2306.07927* (2023).

[34] Wensheng Luo, Zhuo Tang, Yixiang Fang, Chenhao Ma, and Xu Zhou. 2023. Scalable Algorithms for Densest Subgraph Discovery. In *ICDE*. 287–300.

[35] Chenhao Ma, Yixiang Fang, Reynold Cheng, Laks V. S. Lakshmanan, and Xiaolin Han. 2022. A Convex-Programming Approach for Efficient Directed Densest Subgraph Discovery. In *SIGMOD*. 845–859.

[36] Chenhao Ma, Yixiang Fang, Reynold Cheng, Laks V. S. Lakshmanan, Wenjie Zhang, and Xuemin Lin. 2020. Efficient Algorithms for Densest Subgraph Discovery on Large Directed Graphs. In *SIGMOD*. 1051–1066.

[37] Michael Mitzenmacher, Jakub Pachocki, Richard Peng, Charalampos E. Tsourakakis, and Shen Chen Xu. 2015. Scalable Large Near-Clique Detection in Large-Scale Networks via Sampling. In *SIGKDD*. 815–824.

[38] Atsushi Miyauchi and Akiko Takeda. 2018. Robust Densest Subgraph Discovery. In *ICDM*. 1188–1193.

[39] Mark EJ Newman. 2001. Scientific collaboration networks. I. Network construction and fundamental results. *Phys. Rev. E* 64, 1 (2001), 016131.

[40] Lu Qin, Rong-Hua Li, Lijun Chang, and Chengqi Zhang. 2015. Locally Densest Subgraph Discovery. In *SIGKDD*. 965–974.

[41] Barna Saha, Allison Hoch, Samir Khuller, Louiqa Raschid, and Xiao-Ning Zhang. 2010. Dense Subgraphs with Restrictions and Applications to Gene Annotation Graphs. In *RECOMB (Lecture Notes in Computer Science, Vol. 6044)*. 456–472.

[42] Raman Samusevich, Maximilien Danisch, and Mauro Sozio. 2016. Local triangle-densest subgraphs. In *ASONAM*. 33–40.

[43] Ahmet Erdem Sariyüce and Ali Pinar. 2018. Peeling Bipartite Networks for Dense Subgraph Discovery. In *WSDM*. 504–512.

[44] Kelvin Sim, Jinyan Li, Vivekanand Gopalkrishnan, and Guimei Liu. 2009. Mining maximal quasi-bicliques: Novel algorithm and applications in the stock market and protein networks. *Stat. Anal. Data Min.* 2, 4 (2009), 255–273.

[45] Mauro Sozio and Aristides Gionis. 2010. The community-search problem and how to plan a successful cocktail party. In *KDD*. 939–948.

[46] Yizhou Sun, Jiawei Han, Peixiang Zhao, Zhijun Yin, Hong Cheng, and Tianyi Wu. 2009. RankClus: integrating clustering with ranking for heterogeneous information network analysis. In *EDBT (ACM International Conference Proceeding Series, Vol. 360)*. 565–576.

[47] Charalampos E. Tsourakakis. 2015. The K-clique Densest Subgraph Problem. In *WWW*. 1122–1132.

[48] Kai Wang, Xuemin Lin, Lu Qin, Wenjie Zhang, and Ying Zhang. 2020. Efficient Bitruss Decomposition for Large-scale Bipartite Graphs. In *ICDE*. 661–672.

[49] Kai Wang, Xuemin Lin, Lu Qin, Wenjie Zhang, and Ying Zhang. 2022. Towards efficient solutions of bitruss decomposition for large-scale bipartite graphs. *VLDB J.* 31, 2 (2022), 203–226.

[50] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural Graph Collaborative Filtering. In *SIGIR*. 165–174.

[51] Zongyu Xu, Yihao Zhang, Long Yuan, Yuwen Qian, Zi Chen, Mingliang Zhou, Qin Mao, and Weibin Pan. 2023. Effective Community Search on Large Attributed Bipartite Graphs. *Int. J. Pattern Recognit. Artif. Intell.* 37, 02 (2023).

[52] Kaiqiang Yu, Cheng Long, Shengxin Liu, and Da Yan. 2022. Efficient Algorithms for Maximal k-Biplex Enumeration. In *SIGMOD*. 860–873.

[53] Kaiqiang Yu, Cheng Long, Deepak P, and Tanmoy Chakraborty. 2023. On Efficient Large Maximal Biplex Discovery. *IEEE Trans. Knowl. Data Eng.* 35, 1 (2023), 824–829.

[54] Hongyuan Zha, Xiaofeng He, Chris H. Q. Ding, Ming Gu, and Horst D. Simon. 2001. Bipartite Graph Partitioning and Data Clustering. In *CIKM*. 25–32.

[55] Yanxin Zhang, Zhengyu Hua, and Long Yuan. 2024. Top-r Influential Community Search in Bipartite Graphs. *CoRR* abs/2412.06216 (2024).

[56] Yalong Zhang, Rong-Hua Li, Qi Zhang, Hongchao Qin, Lu Qin, and Guoren Wang. 2025. Density Decomposition of Bipartite Graphs. *Proc. ACM Manag. Data* 3, 1 (2025), 30:1–30:25.

[57] Yalong Zhang, Rong-Hua Li, Qi Zhang, and Guoren Wang. 2025. Optimal (α,β)-Dense Subgraph Search in Bipartite Graphs. *CoRR* abs/2508.18616 (2025).

[58] Yuting Zhang, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2021. Pareto-optimal Community Search on Large Bipartite Graphs. In *CIKM*. 2647–2656.

[59] Yuting Zhang, Kai Wang, Wenjie Zhang, Wei Ni, and Xuemin Lin. 2024. Size-bounded Community Search over Large Bipartite Graphs. In *EDBT*. 320–331.

[60] Zuxuan Zhang, Hongxi Li, Kai Huang, and Yuncheng Jiang. 2023. k-bitruss-Attributed Weighted Community Search on Attributed Weighted Bipartite Graphs. In *IEEE BigData*. 562–567.

[61] Keqi Zhou, Junchang Xin, Jinyi Chen, Xian Zhang, Beibei Wang, and Zhiqiong Wang. 2023. Effective and efficient community search with size constraint on bipartite graphs. *Inf. Sci.* 647 (2023), 119511.

[62] Tao Zhou, Jie Ren, Matus Medo, and Yi-Cheng Zhang. 2007. Bipartite network projection and personal recommendation. *Phys. Rev. E* 76, 4 (2007), 046115.

[63] Zhaonian Zou. 2016. Bitruss Decomposition of Bipartite Graphs. In *DASFAA (Lecture Notes in Computer Science, Vol. 9643)*. 218–233.